第6章: Python基礎 — バックエンドの言語（完全版）

> この章では、本アプリの **裏側（バックエンド）** を動かしている言語 **Python（パイソン）** を、完全初心者向けにゼロから学びます。`04` 章で学んだ JavaScript/TypeScript が「ブラウザ側（フロントエンド）」の言語なら、Python は「サーバー側（バックエンド）」の言語です。
>
> 学ぶといっても、文法書を丸暗記するわけではありません。**本アプリの実際のバックエンドコード**（ログイン処理・ログ記録・履歴の差分計算・バックアップの非同期処理・データの設計図）を **1行ずつ、本物のまま** 読み解きながら、必要な文法をその場で身につけます。「この記号は何？」「この書き方は何のため？」を、実コードの上で全部つぶしていきます。
>
> この章を読み終えると、`kosu/` フォルダの `.py` ファイルを開いたとき、**書いてあることが読める** ようになります。それが次章「07_Django_バックエンド.md」で、Webサーバーの仕組みを理解する土台になります。

この章で学ぶこと

- **Pythonとは何か** — どこで動き、本アプリのどこにいるのか
- **インデント（字下げ）** — Pythonでは空白が「意味」を持つ。最重要ルール
- **変数とデータ型** — 数値・文字列・真偽値・None
- **リスト・辞書・タプル** — 複数のデータをまとめる3つの入れ物
- **条件分岐 `if` / 繰り返し `for`** — プログラムの流れを作る
- **リスト内包表記** — リストを1行で作る省略記法（実コードに登場）
- **関数 `def`** — 処理に名前を付けて使い回す。引数・初期値・戻り値
- **クラスと `self` / `__init__` / `__str__` / 継承** — 設計図からモノを作る考え方
- **例外処理 `try-except`** — エラーで止まらないようにする
- **`with` 文** — ファイルを安全に開いて確実に閉じる
- **`import` と相対import** — 他のファイルの機能を借りる
- **三項演算子・f文字列・デコレータ `@`** — 実コードに頻出する記法
- **実コード完全解説** — `main_views.py`・`main_utils.py`・`asynchronous_views.py`・`signals.py`・`models.py` の本物のコードを1行ずつ

目次

0. 前提知識：Pythonはどこで動くのか
1. インデント — Python最大の特徴
2. 変数とデータ型
3. リスト・辞書・タプル

4. 条件分岐 if
5. 繰り返し for
6. リスト内包表記
7. 関数 def（引数・初期値・戻り値）
8. 三項演算子・f文字列
9. クラスとインスタンス（self / **init** / **str** / 継承）
10. 例外処理 try-except
11. with 文（ファイルを安全に扱う）
12. import と相対import
13. デコレータ @
14. 実コード全解説①：main_views.py の Login・PrintToLogger・get_logs
15. 実コード全解説②：main_utils.py の検証関数
16. 実コード全解説③：asynchronous_views.py の validate_dates
17. 実コード全解説④：models.py の member クラスの形
18. 実コード全解説⑤：signals.py の差分計算
19. トラブルシューティング
20. 演習問題
21. この章のまとめ

---

0. 前提知識：Pythonはどこで動くのか

> **Python（パイソン：プログラミング言語の名前）とは？** 1991年に登場した、**読みやすさ** を重視したプログラミング言語。英語に近い書き方ができ、世界中で「最初に学ぶ言語」として人気です。名前はイギリスのコメディ番組「空飛ぶモンティ・パイソン」に由来し、ヘビとは関係ありません（ただしロゴはヘビ）。

0.1 フロントエンドとバックエンドの違い

本アプリは2つの世界に分かれています。

	動く場所	言語	役割	本アプリのフォルダ
フロントエンド	利用者のブラウザ（PC）	TypeScript/React	画面の表示・入力受付	`frontend/`
バックエンド	サーバー（会社のコンピュータ）	**Python/Django**	データの保存・計算・判定	`kosu/`・`hozen_another/`

> **たとえ話：レストラン** フロントエンド（ブラウザ）は **客席とメニュー** です。お客さん（利用者）が見て、注文（入力）します。バックエンド（Python）は **厨房** です。注文を受け取り、料理を作り（データを処理し）、冷蔵庫（データベース）から材料を出し入れします。お客さんは厨房を直接見ませんが、厨房がなければ料理は出てきません。**この章で学ぶPythonは、その厨房の言葉** です。

0.2 本アプリのPythonファイルの場所

`kosu/` フォルダの中に、本アプリのバックエンドの頭脳が入っています。

コピー

```
kosu/
├── models.py            # データの「設計図」（テーブルの形）。§17で解説
├── signals.py           # データ変更を自動で記録する仕掛け。§18で解説
├── urls.py              # どのURLでどの処理を呼ぶか（07章）
├── views/               # 各機能の処理本体
│   ├── main_views.py    # ログイン・メニュー・ログ。§14で解説
│   ├── asynchronous_views.py  # バックアップ等の重い処理。§16で解説
│   └── ...
└── utils/
    └── main_utils.py    # 便利な共通部品（検証関数など）。§15で解説
```

> **拡張子 `.py`** が付くファイルが Python のファイルです。中身は **ただの文字（テキスト）** で、メモ帳でも開けます。これを Python が「上から順に」読んで実行します。

0.3 Pythonの実行のされ方（おおまかに）

Python は **インタプリタ（interpreter：通訳）型** の言語です。書いたコードを、Python本体（通訳）が **1行ずつその場で翻訳しながら** 実行します。

コピー

```
あなたが書いた .py ファイル（人間の言葉に近い）
        ↓
Python本体（python.exe）が1行ずつ読んで
        ↓
コンピュータが分かる命令に翻訳しながら実行
```

> **なぜ？** C言語などは、実行前に全体をまとめて機械語に翻訳（コンパイル）します。Pythonは翻訳しながら実行するので、**書いてすぐ試せる** 手軽さがあります。本アプリでは `python manage.py runserver` というコマンドで、Pythonがバックエンドを起動します（`01`・`07` 章）。

---

1. インデント ― Python最大の特徴

Pythonを学ぶうえで **絶対に最初に押さえるべき** ルールがこれです。他の多くの言語と決定的に違う点なので、ここを外すと一切動きません。

1.1 インデントとは

> **インデント（indent：字下げ）とは？** 行の先頭に入れる **空白（スペース）** のこと。文章を書くとき段落の最初を1マス下げるのと同じ「字下げ」です。

JavaScript では、処理のまとまり（ブロック）を **波カッコ** `{ }` で囲みました（`04` 章）。

コピー

```
// ※説明用の簡易例（JavaScript）
if (x > 0) {
  // { } の中が「if が成立したときの処理」
  console.log("正の数");
}
```

Python には波カッコがありません。 代わりに **インデント（字下げ）の深さ** で「どこからどこまでが1つのまとまりか」を表します。

> **▼ このコードがやること（先に日本語で）**：x が 0 より大きいとき、「正の数」と表示する。Python版です。

コピー

```
# ※説明用の簡易例（Python）
# 行末に ：（コロン）を付ける
if x > 0:
    # 字下げされた行が「if が成立したときの処理」
    print("正の数")
```

- `if x > 0:` … 条件。**行末に** `:` **（コロン）** を付けるのがPythonの約束。
- 次の行 `print(...)` … **先頭が字下げ** されている。この「字下げされたまとまり」が if の中身。

1.2 字下げの深さで「中・外」が決まる

> **▼ このコードがやること:** 字下げの深さが変わると、処理のまとまりが変わることを示します。

コピー

```
# ※説明用の簡易例
if x > 0:
    # 字下げあり → if の中（x>0 のときだけ実行）
    print("A")
    # 字下げあり → これも if の中
    print("B")
# 字下げなし → if の外（いつでも実行）
print("C")
```

- `A`・`B` は字下げされているので **if の中**。`x > 0` のときだけ表示。
- `C` は字下げが戻って **if の外**。`x` の値に関係なく常に表示。

> ⚠️ **インデントは「揃える」こと。混ぜると即エラー**: Pythonの字下げは **半角スペース** で行います。本アプリは **スペース2個** を1段の字下げにしています（本章のコード例も2個です。本物のソースもそう）。一般的には4個が多いですが、**1つのファイルの中で統一されていれば** 何個でも動きます。ただし **スペースとタブ（Tabキー）を混ぜると** `IndentationError`（インデントエラー）で止まります。エディタの設定で「Tabはスペースに変換」にしておくのが安全です。
>
> **なぜPythonはわざわざこんなルールにしたの？** 波カッコを使う言語でも、人間が読みやすいように結局インデントを揃えます。「だったらインデントそのものを文法にしてしまえば、誰が書いても見た目が揃って読みやすい」という思想です。実際、Pythonのコードはどの人が書いても似た見た目になります。

1.3 本物のコードで確認

本アプリ `main_views.py` の冒頭近くにある、実際のコードを見てみましょう（§14で全解説しますが、まずインデントだけ注目）。

コピー

```
# クラス定義（一番外側、字下げ0）
class PrintToLogger:
  # メソッド定義（字下げ1段＝クラスの中）
  def write(self, message):
    # if文（字下げ2段＝メソッドの中）
    if message.strip():
      # if の中身（字下げ3段）
      views_logger.info(message)
```

字下げが深くなるほど「内側」です。

- `class PrintToLogger:` … 字下げ0。一番外側。
- `def write(...):` … 字下げ1段（スペース2個）。クラスの **中**。
- `if message.strip():` … 字下げ2段（スペース4個）。メソッドの **中**。

- `views_logger.info(message)` … 字下げ3段（スペース6個）。if の **中**。

この「コロン `:` で始まり、次の行から字下げで中身を書く」が、Pythonの全構文（if・for・def・class・try・with…）に共通する基本形 です。この章で何度も出てきます。

---

2. 変数とデータ型

2.1 変数 — 値に名前を付ける箱

> **変数（へんすう：値を入れておく名前付きの箱）とは？** データに名前を付けて、後で使えるようにしておく仕組み。

Pythonの変数は、JavaScriptの `let` や `const` のような **宣言キーワードが不要** です。いきなり `名前 = 値` で作れます。

> **▼ このコードがやること:** 3つの変数に値を入れる。

コピー

```
# ※説明用の簡易例
# input_number という箱に 12345 を入れる
input_number = 12345
# name という箱に "admin"（文字）を入れる
name = "admin"
# authority という箱に True（真）を入れる
authority = True
```

- `=`（イコール）は「等しい」ではなく「**右の値を左の箱に入れる（代入）**」という意味。算数の `=` とは違うので注意。
- 「等しいか？」を調べたいときは `==`（イコール2つ）を使います（§4）。

> **用語：代入（だいにゅう）** 変数に値を入れること。`x = 5` は「x に 5 を代入する」と読みます。

本物のコード（`main_views.py` の Login）でも、同じ書き方が出てきます。

コピー

```
# data という箱に、解析したデータを入れる
data = json.loads(request.body)
# input_number に、その中の従業員番号を入れる
input_number = data.get('employee_no')
```

2.2 データ型 — 値の種類

変数に入れる値には **種類（型）** があります。Pythonの基本的な型は次の通り。

型（よみ）	意味	例	本アプリでの登場例
`int`（イント）	整数	`12345`, `-3`, `0`	従業員番号
`float`（フロート）	小数	`3.14`, `0.5`	（計算結果など）
`str`（ストリング）	文字列	`"admin"`, `'P'`	氏名・ショップ
`bool`（ブール）	真偽値（True/False）	`True`, `False`	権限・管理者フラグ
`None`（ノン）	「値がない」を表す特別な値	`None`	未設定の項目

用語：文字列（もじれつ／string） 文字の並び。Pythonでは **シングルクォート `'...'` でもダブルクォート `"..."` でも** 囲めます（どちらでも同じ）。本アプリは両方混在しています。

用語：真偽値（しんぎち／boolean）「はい/いいえ」「オン/オフ」のような2択を表す値。Pythonでは **`True`（真）** と **`False`（偽）** の2つだけ。**先頭が大文字** なのが特徴（JavaScriptは小文字 `true`）。

用語：`None`（ノン）「何も入っていない」「該当なし」を表すPython専用の特別な値。JavaScriptの `null` に相当します。データベースで「未入力」の項目はこの `None` になります。

2.3 型の自動判別と変換

Pythonは、入れた値から型を **自動で判断** します（型を宣言する必要がない）。

コピー

```
# ※説明用の簡易例
# 5 は整数なので x は int 型
x = 5
# "5" は文字なので x は str 型（同じxでも型が変わる）
x = "5"
```

⚠️ **「5」と「"5"」は別物:** `5`（数値）と `"5"`（文字）は見た目は似ていますが **まったく別の型** です。`5 + 3` は `8` ですが、`"5" + "3"` は文字をつなげた `"53"` になります。だから本アプリの `main_utils.py` では、文字で来た従業員番号を `int(value)` で **数値に変換** してから比較しています（§15で実コード解説）。

型を変換する関数：

コピー

```
# ※説明用の簡易例
# 文字 "123" → 数値 123
int("123")
# 数値 123 → 文字 "123"
str(123)
```

---

3. リスト・辞書・タプル

データを **1個ずつ** ではなく **まとめて** 扱う入れ物が3種類あります。本アプリのコードに全部出てきます。

3.1 リスト（list）— 順番付きの並び

> **リスト（list：複数の値を順番に並べた入れ物）とは？** 値を `[ ]`（角カッコ）で囲み、カンマで区切って並べたもの。買い物リストのように「順番がある一列の箱」です。
>
> **▼ このコードがやること:** 3つの数字を1つのリストにまとめ、中身を取り出す。

コピー

```
# ※説明用の簡易例
# 角カッコで囲んだリスト
numbers = [10, 20, 30]
# → 10 （0番目。番号は0から始まる！）
print(numbers[0])
# → 20 （1番目）
print(numbers[1])
# → [10, 20, 30, 40] 末尾に追加
numbers.append(40)
```

- `[ ]` … リストの目印。
- `numbers[0]` … **0番目** の要素。Pythonは **0から数えます**（1番目ではなく0番目が先頭）。
- `.append(値)` … リストの末尾に値を1つ追加する命令。

本物のコード（`main_views.py` の TeamMenu）でも、リストが活躍します。

コピー

```
# 空のリストを作る
follow_message_list = []
...
# メッセージを末尾に追加していく
follow_message_list.append(follow_message)
```

> 最初は空っぽ `[]` のリストを作り、ループの中で `.append()` で1つずつ詰めていく——この「空リストを作って詰める」パターンは超頻出です（§5で再登場）。

3.2 辞書（dict）— 名前と値のペア

> **辞書（dict：キーと値をペアで持つ入れ物）とは？** `{ }`（波カッコ）で囲み、「**キー（名前）：値**」のペアを並べたもの。国語辞典で「単語（キー）→意味（値）」を引くのと同じ。
>
> ▼ **このコードがやること:** 名前と従業員番号のペアを辞書にまとめ、名前で値を引く。

コピー

```
# ※説明用の簡易例
# キー:値 のペアを波カッコで
person = {"name": "admin", "no": 12345}
# → "admin"  ("name"というキーで引く)
print(person["name"])
# → 12345
print(person["no"])
```

- `{ }` … 辞書の目印（リストの `[ ]` と区別！）。
- `"name": "admin"` … キー `"name"` に値 `"admin"` を結びつける。
- `person["name"]` … キーを指定して値を取り出す。

本物のコード（`main_views.py` の manifest）はまさに辞書そのものです。

コピー

```
data = {
  # キー "name" → 値 "業務工数システム"
  "name": "業務工数システム",
  # キー "short_name" → 値 "業務工数"
  "short_name": "業務工数",
  # キー "start_url" → 値 "/"
  "start_url": "/",
}
```

> **リストと辞書の使い分け:** 「順番で取り出したい」ならリスト（ `numbers[0]` ）、「名前で取り出したい」なら辞書（ `person["name"]` ）。本アプリのAPIが返すデータは、ほぼこの辞書の形（JSON）です（ `07` 章）。

辞書から安全に値を取る `.get()` ：

コピー

```
# ※説明用の簡易例
# → "admin"（キーがあれば値）
person.get("name")
# → None（キーがなくてもエラーにならず None を返す）
person.get("age")
```

> **`person["age"]` と `person.get("age")` の違い:** 角カッコ `["age"]` はキーが無いと **エラーで止まる**。 `.get("age")` はキーが無くても **None を返して止まらない**。本アプリは「あるか分からない値」に `.get()` を多用します（例： `data.get('employee_no')` ）。

3.3 タプル（tuple）— 変更できないリスト

> **タプル（tuple：あとから変更できない並び）とは？** `( )` （丸カッコ）で囲んだ、リストに似た並び。ただし **一度作ったら中身を変えられない**（固定）。

コピー

```
# ※説明用の簡易例
# 丸カッコ。中身は固定
point = (10, 20)
# → 10 （取り出しはリストと同じ）
print(point[0])
# point[0] = 99          # ← これはエラー（変更不可）
```

本物のコード（ `models.py` の member）では、選択肢の固定リストがタプルの並びで定義されています。

コピー

```
shop_list = [
  # ('表示値', '保存値') のタプル
  ('P', 'P'),
  ('R', 'R'),
  ('W1', 'W1'),
  ...
]
```

- 外側は `[ ]`（リスト）、中の各要素 `('P', 'P')` が **タプル**。
- 「リストの中にタプルが並ぶ」構造です。Djangoの選択肢（プルダウンの候補）はこの形で書く約束（§17）。

本物のコード（`asynchronous_views.py`）では、関数に渡す引数をタプルでまとめています。

コピー

```
# 2つの値をタプルにまとめる
args = (start_day, end_day)
```

> **タプルとリストの使い分け:**「あとで変わらないもの・変わってほしくないもの」はタプル `( )`、「追加・削除する一覧」はリスト `[ ]`。座標・選択肢・固定の組はタプル、という感覚です。

---

4. 条件分岐 if

4.1 if / elif / else の基本形

> **条件分岐（じょうけんぶんき）とは？**「もし〜なら A、そうでなければ B」と、状況によって処理を変えること。

コピー

```
# ※説明用の簡易例
# もし score が 80 以上なら
if score >= 80:
    print("合格")
# そうでなく、60以上なら（elif = else if）
elif score >= 60:
    print("追試")
# どれでもなければ
else:
    print("不合格")
```

- `if 条件:` … 最初の条件。**末尾に `:`**、次の行は字下げ（§1）。
- `elif 条件:` …「そうでなく、もし〜なら」。`else if` の略。いくつでも並べられる。
- `else:` …「どれにも当てはまらなければ」。条件は書かない。

> ⚠️ **`=` と `==` の違い（最頻出ミス）**：`=` は **代入**（箱に入れる）、`==` は **比較**（等しいか調べる）。`if x = 5:` はエラー。`if x == 5:` が正解。

4.2 比較の記号（比較演算子）

記号	意味	例
`==`	等しい	`x == 5`
`!=`	等しくない	`x != 5`
`>` `>=`	より大きい／以上	`x >= 80`
`<` `<=`	より小さい／以下	`x < 0`

本物のコード（`asynchronous_views.py` の validate_dates）の比較：

コピー

```
# 終了日が今日以降なら
if end_date_obj >= today_date_obj:
    ...
# 開始日が終了日より後なら
if start_date_obj > end_date_obj:
    ...
```

4.3 条件をつなぐ（and / or / not）

記号	意味	たとえ
`and`	かつ（両方true）	「20歳以上 **かつ** 会員」
`or`	または（どちらかtrue）	「現金 **または** カード」
`not`	否定（true↔false反転）	「会員 **でない**」

コピー

```
# ※説明用の簡易例
# x が 0 より大きく、かつ 100 より小さい
if x > 0 and x < 100:
    print("範囲内")
```

本物のコード（`asynchronous_views.py`）：

コピー

```
# start_day が無い、または end_day が無いなら
if not start_day or not end_day:
    return JsonResponse(...)
```

- `not start_day` … 「start_day が空（=なし）なら」という意味。
- `or` … どちらか一方でも空ならエラー、という判定。

> **用語：truthy（トゥルーシー）/ falsy（フォルシー）** Pythonでは、`if 変数:` のように直接書くと、変数が「実質的に空っぽか」で真偽が決まります。空文字 `""`・`0`・`None`・空リスト `[]` は **falsy（偽とみなす）**、それ以外は **truthy（真とみなす）**。だから `if not start_day:` は「start_day が空文字や None なら」という意味になります。本物のコード `main_utils.py` の `if not value:`（§15）も同じ仕組みです。

4.4 存在チェック（in / is）

コピー

```
# ※説明用の簡易例
# 辞書 person に "name" というキーがあるか
if "name" in person:
    ...
# old_instance が None かどうか（is None で判定）
if old_instance is None:
    ...
```

> `== None` ではなく `is None`：None かどうかの判定は、Pythonの慣習として `is None` を使います（`signals.py` でも `if old_instance:` の形で多用）。`is` は「まったく同じものか」を見る演算子です。

---

5. 繰り返し for

5.1 for の基本 — リストを順番に処理

> **繰り返し（ループ）とは？** 同じ処理を、対象を変えながら何度も実行すること。「リストの中身を1個ずつ取り出して処理する」のが典型。
>
> **▼ このコードがやること:** リストの中の数字を1つずつ取り出して表示する。

コピー

```
# ※説明用の簡易例
numbers = [10, 20, 30]
# numbers から1つずつ取り出して n に入れる
for n in numbers:
    # → 10, 20, 30 が順に表示される
    print(n)
```

- `for 変数 in リスト:` … リストから1個ずつ取り出し、`変数` に入れて、字下げした中身を実行。これをリストの数だけ繰り返す。
- `n` は1回目10、2回目20、3回目30、と中身が変わります。

5.2 本物のコードで見る for

`main_views.py` の TeamMenu には、ループの実例が詰まっています。

コピー

```
# 班員の番号を1つずつ m に
for m in valid_member_numbers:
  follow_message = ''
  # その番号の人がいたら
  if member.objects.filter(employee_no=m).exists():
    member_name = member.objects.get(employee_no=m).name
    # 過去7日を1日ずつ d に（ループの中のループ）
    for d in day_list:
      ...
```

- 外側の `for m in valid_member_numbers:` … 班員リストを1人ずつ処理。
- 内側の `for d in day_list:` … その人について、7日分を1日ずつ処理。
- **ループの中にループ（入れ子）** になっています。字下げの深さで「どっちのループの中か」が分かります（§1のインデントがここで効く）。

5.3 range（連番を作る）

> **▼ このコードがやること:** 1から7までの連番を使って、過去7日分の日付を作る。

`main_views.py` の TeamMenu の実コード：

コピー

```
day_list = [today - datetime.timedelta(days=d) for d in range(1, 8)]
```

- `range(1, 8)` … `1, 2, 3, 4, 5, 6, 7` という連番を作る（**8は含まない**。「1以上8未満」）。
- これは「リスト内包表記」という1行の書き方です。次の§6で詳しく解説します。

> ⚠️ **`range(1, 8)` は8を含まない:** Pythonの `range(開始, 終了)` は **終了の手前まで**。`range(1, 8)` は1〜7の7個。「7日分ほしいから 1〜7」で `range(1, 8)` になっています。

5.4 ループを止める break

コピー

```
# 本物のコード（main_views.py TeamMenu より）
for d in day_list:
  if Business_Time_graph.objects.filter(employee_no3=m, work_day2=d).exists():
    ...
    if not kosu_get.judgement:
      follow_message = f'{member_name}氏の工数未入力があります。'
      follow_message_list.append(follow_message)
      # 1件見つけたら、このループを途中で抜ける
      break
```

- `break` … ループを **途中で打ち切る** 命令。「未入力が1件でも見つかれば、もう調べる必要はない」のでループを抜けます。無駄な処理をしないための工夫です。

---

6. リスト内包表記

Pythonらしい、**リストを1行で作る** 省略記法です。実コードに出てくるので必ず読めるようにします。

6.1 基本の形

> **リスト内包表記（ないほうひょうき／list comprehension）とは？** `[ 式 for 変数 in リスト ]` の形で、新しいリストを1行で作る書き方。

普通に書くと（for で1個ずつ append）：

コピー

```
# ※説明用の簡易例（普通の書き方）
# 空リスト
squares = []
# 1個ずつ取り出して
for n in [1, 2, 3]:
    # 2乗を追加
    squares.append(n * n)
# squares は [1, 4, 9]
```

内包表記で書くと（同じ結果が1行に）：

コピー

```
# ※説明用の簡易例（内包表記）
# [1, 4, 9]
squares = [n * n for n in [1, 2, 3]]
```

読み方は「**右から左**」：「`[1, 2, 3]` から `n` を1個ずつ取り、`n * n` を計算して、リストにする」。

6.2 条件付きの内包表記

`if` を後ろに付けると、条件に合うものだけ集められます。

> ▼ **このコードがやること:** 班員番号のリストから、「空でない」番号だけを集める。

本物のコード（`main_views.py` の TeamMenu）：

コピー

```
valid_member_numbers = [
  num for num in member_numbers if num is not None and num != ''
]
```

読み方： - `member_numbers` から `num` を1個ずつ取り出す。- `if num is not None and num != ''` … 「None でなく、かつ空文字でもない」場合だけ。- それを集めて `valid_member_numbers` という新しいリストにする。

つまり「**登録されている班員番号だけを抜き出したリスト**」を1行で作っています。普通に書くと5行くらいになるものが1行で済みます。

6.3 日付リストの内包表記（実コード）

§5.3で出た日付リストも内包表記でした。

コピー

```
day_list = [today - datetime.timedelta(days=d) for d in range(1, 8)]
```

読み方： - `range(1, 8)` から `d` を1個ずつ（1〜7）取り出す。- `today - datetime.timedelta(days=d)` … 今日から d 日前の日付を計算。- それを集めて `day_list`（昨日〜7日前の7個の日付リスト）にする。

> **用語： `datetime.timedelta(days=d)`** 「d日分の時間の長さ」を表すもの。`today - timedelta(days=1)` で「昨日」が求まります。日付の足し算・引き算に使います。

6.4 別の内包表記（models.py）

`models.py` の def_choice クラスにも、文字列を1文字ずつ回す内包表記があります。

コピー

```
def_list = [(x, x) for x in "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwx$"]
```

- 文字列も for で **1文字ずつ** 取り出せます（ `x` に 'A'、'B'、...と順に入る）。
- `(x, x)` … 各文字を「(表示値, 保存値)」のタプルにする。
- 結果は `[('A','A'), ('B','B'), ...]` という選択肢リスト。**1文字ずつ手で書く手間を省いて** います。

---

7. 関数 def（引数・初期値・戻り値）

7.1 関数とは

> **関数（かんすう／function）とは？** ひとまとまりの処理に **名前を付けて**、いつでも呼び出せるようにしたもの。「材料を入れると料理を返す自動販売機」のイメージ。
>
> **▼ このコードがやること:** 2つの数を受け取り、その合計を返す関数を定義して使う。

コピー

```
# ※説明用の簡易例
# add という名前の関数を定義。a と b を受け取る
def add(a, b):
    # 計算
    result = a + b
    # 結果を返す
    return result

# 関数を呼び出す。answer は 8
answer = add(3, 5)
```

- `def 関数名(引数):` … 関数の定義。`def` は define（定義する）の略。**末尾に** `:`、中身は字下げ（§1）。
- `a, b` … **引数（ひきすう）**。関数に渡す材料。
- `return 値` … **戻り値（もどりち）** を返して関数を終える。`return` で返した値が、呼び出した側（ `add(3,5)` ）の結果になる。
- `add(3, 5)` … 関数の **呼び出し**。`a=3, b=5` として実行される。

> **用語：引数（ひきすう）** 関数に渡す入力。料理の「材料」。**用語：戻り値（もどりち／返り値）** 関数が返す結果。料理の「完成品」。

7.2 引数の初期値（デフォルト引数）

引数に **あらかじめ値を設定** しておくと、呼び出すとき省略できます。

コピー

```
# ※説明用の簡易例
# greeting に初期値
def greet(name, greeting="こんにちは"):
    print(greeting + name + "さん")

# → こんにちは田中さん（greeting 省略 → 初期値が使われる）
greet("田中")
# → おはよう田中さん（指定すればそちら）
greet("田中", "おはよう")
```

- `greeting="こんにちは"` … 引数に **初期値（デフォルト値）** を設定。
- 呼び出し時に省略すると初期値、指定すればその値が使われる。

本物のコード（`main_views.py` の AdministratorKosuList）でも、リクエストから値を取るときに初期値を使っています。

コピー

```
# mode が無ければ 'day'
mode = request.query_params.get('mode', 'day')
```

- `.get('mode', 'day')` …「mode を取る。もし無ければ初期値 `'day'`」。これも初期値の考え方です。

7.3 引数なし・戻り値なしの関数

関数は引数や戻り値が無くても作れます。

本物のコード（`main_views.py` の get_logs。§14で全解説）：

コピー

```
# request だけ受け取る
def get_logs(request):
  with open('web_console.log', 'r') as log_file:
    logs = log_file.readlines()
  # JSONレスポンスを返す
  return JsonResponse({'logs': logs})
```

本物のコード（`main_utils.py` の get_all_model_names_in_myapp。§15で全解説）は引数なし：

コピー

```
# 引数なし（カッコの中が空）
def get_all_model_names_in_myapp():
  model_names = []
  ...
  return model_names
```

7.4 複数の値を返す（タプルで返す）

Pythonの関数は、カンマで区切ると **複数の値を一度に返せます**（実体はタプル §3.3）。

本物のコード（`main_utils.py` の validate_employee_no_logic。§15で全解説）：

コピー

```
def validate_employee_no_logic(value, member_model):
  if not value:
    # 2つの値（True と None）を返す
    return True, None
  ...
  # 2つの値を返す
  return False, 'は自然数で入力して下さい'
```

- `return True, None` ...「成功したか（True/False）」と「メッセージ」の **2つを同時に返す**。
- 受け取る側は `成功, メッセージ = validate_...()` のように2つの変数で受けます。「OKか」と「理由」をセットで返す定番パターンです。

---

8. 三項演算子・f文字列

実コードに頻出する、2つの便利な記法です。

8.1 三項演算子（条件を1行で書く）

> **三項演算子（さんこうえんざんし）とは？** if-else を1行で書く省略記法。`A if 条件 else B` の形で「条件が真なら A、偽なら B」。

普通に書くと：

コピー

```
# ※説明用の簡易例（普通の if）
if score >= 60:
    result = "合格"
else:
    result = "不合格"
```

三項演算子で書くと（1行）：

コピー

```
# ※説明用の簡易例（三項演算子）
result = "合格" if score >= 60 else "不合格"
```

読み方：「`"合格"` を入れる、もし `score >= 60` なら。そうでなければ `"不合格"`」。値が先、条件が後、という英語の語順です。

本物のコード（`main_views.py` の TeamMenu）：

コピー

```
team_get = team_filter.first() if team_filter.exists() else None
```

- 「team_filter にデータがあれば最初の1件、なければ None」を team_get に入れる。

本物のコード（`main_views.py` の AdministratorKosuList）：

コピー

```
judgement = True if search_judgement == 'OK' else False
```

- 「search_judgement が 'OK' なら True、そうでなければ False」。

本物のコード（`signals.py`）でも多用：

コピー

```
session_data = request.session.get('login_No') if request else None
```

- 「request があればそのログイン番号、なければ None」。

8.2 f文字列（文字の中に値を埋め込む）

> **f文字列（エフもじれつ／f-string）とは？** 文字列の前に `f` を付け、`{ }` の中に変数や式を書くと、その値が文字に埋め込まれる記法。

コピー

```
# ※説明用の簡易例
name = "田中"
# → "田中さん、こんにちは"
message = f"{name}さん、こんにちは"
```

- 先頭の `f` が目印。
- `{name}` … この部分が変数 name の中身（"田中"）に置き換わる。
- `f` を付け忘れると、`{name}` がそのまま文字として出てしまうので注意。

本物のコード（`main_views.py` の TeamMenu）：

コピー

```
follow_message = f'{member_name}氏の工数未入力があります。'
```

- `{member_name}` が実際の名前（例「admin」）に置き換わり、「admin氏の工数未入力があります。」になる。

本物のコード（`models.py` の `__str__`）でも頻出：

コピー

```
return f'{self.created_at} on {self.status} (TaskID: {self.task_id})'
```

- 3つの値（作成日時・状態・タスクID）を1つの文に埋め込んでいます。

> **`{ }` の中には式も書ける:** `f"合計は{a + b}円"` のように、計算式も書けます。`asynchronous_views.py` には `f'ファイル書き込みエラー: {str(e)}'` のように、変換関数 `str(e)` を埋め込んだ例があります。

---

9. クラスとインスタンス（self / init / str / 継承）

ここはPythonの中でも少し抽象的ですが、本アプリは **クラスだらけ** なので、しっかり理解します。

9.1 クラスとは「設計図」、インスタンスとは「製品」

> **クラス（class：設計図）とは？**「データ（属性）」と「処理（メソッド）」をひとまとめにした **設計図**。**インスタンス（instance：実体）とは？** 設計図から作られた **実際のモノ**。
>
> **たとえ話：たい焼き クラス** は「たい焼きの **金型**」。形（属性）と焼き方（メソッド）が決まっています。**インスタンス** は、その金型で焼いた **1個1個のたい焼き**。あんこ入り・クリーム入りなど中身（データ）は違っても、形は同じ。1つの金型（クラス）から、何個でもたい焼き（インスタンス）を作れます。
>
> **▼ このコードがやること:** 「人」の設計図（クラス）を作り、そこから田中さんという実体（インスタンス）を作る。

コピー

```
# ※説明用の簡易例
# 「人」の設計図（クラス）
class Person:
    # 作るときに呼ばれる初期化メソッド
    def __init__(self, name, age):
        # この人の名前を記録
        self.name = name
        # この人の年齢を記録
        self.age = age

    # この人ができること（メソッド）
    def greet(self):
        print(f"私は{self.name}です")

# 設計図から「田中さん」を作る（インスタンス化）
tanaka = Person("田中", 30)
# → 私は田中です
tanaka.greet()
```

> **用語：メソッド（method）** クラスの中に書かれた関数。「そのモノができること（動作）」。`greet`（あいさつする）がメソッド。普通の関数との違いは、第1引数が `self`（自分自身）であること。

9.2 `__init__`（初期化メソッド）

> **`__init__`（イニット）とは？** インスタンスを作る瞬間に **自動で呼ばれる** 特別なメソッド。「作るときの初期設定」を書く場所。前後のアンダースコア2個 `__` が「特別なメソッド」の目印（「ダンダー」と読みます）。

`Person("田中", 30)` と書くと、自動で `__init__(self, "田中", 30)` が呼ばれ、名前と年齢がセットされます。

本物のコード（`main_utils.py` の CustomPagination）の `__init__`：

コピー

```
class CustomPagination(PageNumberPagination):
  page_size = 20

  # 作られるとき自動実行
  def __init__(self):
    last_record = administrator_data.objects.order_by("id").last()
    if last_record is not None:
      # 1ページの表示件数を設定から取得
      self.page_size = last_record.menu_row
```

- このページ送り部品（§15で詳説）は、作られた瞬間に **管理設定から「1ページに何件表示するか」を読み込んで** 自分にセットします。

9.3 `self`（自分自身）

> **`self`（セルフ：自分自身）とは？** メソッドの中で「このインスタンス自身」を指す名前。たい焼きの例なら「**この** たい焼き」。

- メソッドの **第1引数は必ず** `self` にする約束（Pythonのルール）。
- `self.name` は「このインスタンスの name」。`tanaka.name` なら "田中"、`suzuki.name` なら "鈴木" と、インスタンスごとに別の値を持てます。

> **なぜ self が必要？** 1つのクラスから何個もインスタンスを作るので、「どのインスタンスの name か」を区別する必要があります。`self` が「今まさに処理している、その個体」を指します。**呼び出すときは self を書きません**（`tanaka.greet()` であって `tanaka.greet(tanaka)` ではない）。Pythonが自動で渡してくれます。

本物のコード（`main_views.py` の PrintToLogger）：

コピー

```
class PrintToLogger:
  # 第1引数は self（自分自身）
  def write(self, message):
    if message.strip():
      views_logger.info(message)

  # こちらも第1引数は self
  def flush(self):
    pass
```

9.4 `__str__`（文字列表現）

> **`__str__`（ストア）とは？** そのインスタンスを **文字として表示したいとき** に自動で呼ばれる特別なメソッド。`print(インスタンス)` したときに「何と表示するか」を決めます。

本物のコード（`models.py` の member）：

コピー

```
class member(models.Model):
  ...
  def __str__(self):
    # この人を表示するときは「氏名」を出す
    return self.name
```

- これにより、Djangoの管理画面などで member インスタンスを表示すると、idの数字ではなく **氏名** が出ます。人間に分かりやすくする工夫です。

本物のコード（`models.py` の Business_Time_graph）はもっと凝った `__str__`：

コピー

```
def __str__(self):
  return str(self.id) + '__' + str(self.work_day2) + ':' + str(self.employee_no3)
```

- `str(...)` で数値や日付を文字に変換し、`+` でつなげて「id__就業日:従業員番号」という1つの文字列を作っています。
- 例：「`42__2026-06-01:12345`」のような表示になります。

> **用語：`+` による文字列連結** 文字列同士を `+` でつなげると、くっついた1つの文字列になります（`"あ" + "い"` → `"あい"`）。ただし **数値と文字はそのまま `+` できない** ので、`str(self.id)` で数値を文字に変えてからつなげています。

9.5 継承（けいしょう）— 設計図を引き継ぐ

> **継承（inheritance）とは？** 既存のクラス（親）の機能を **引き継いで**、新しいクラス（子）を作ること。「親の能力をそのまま受け継ぎ、必要な部分だけ追加・変更する」。
>
> **たとえ話：** 「乗り物」という親クラスに「走る」機能があるとき、「車」クラスは乗り物を継承して「走る」をタダで手に入れ、さらに「クラクションを鳴らす」を追加する、というイメージ。

書き方は `class 子クラス(親クラス):`。

本物のコード（`models.py`）— すべてのモデルは `models.Model` を継承：

コピー

```
# models.Model を継承
class member(models.Model):
  ...
```

- `(models.Model)` … Djangoが用意した「データベースのテーブルになれる」親クラスを継承。
- これだけで、member は **データベースに保存・検索・削除する機能をタダで** 手に入れます（`.objects.get()` など）。自分では書いていないのに使えるのは、親から継承しているからです（`07` 章で詳説）。

本物のコード（`main_utils.py`）— ページ送り部品も継承：

コピー

```
# 既製のページ送り機能を継承
class CustomPagination(PageNumberPagination):
  page_size = 20
  def __init__(self):
    # 一部だけ自分用にカスタマイズ
    ...
```

> **継承で出てくる `super()`**：子クラスから「親の同名メソッド」を呼ぶときに使います。本物のコード（`models.py` の AsyncTask）： `python def save(self, *args, **kwargs): super().save(*args, **kwargs) # まず親（Model）の保存処理を実行 ... # その後、自分の追加処理（古いレコード削除）` 「親の保存をやってから、自分独自の後始末を足す」という流れです。`*args, **kwargs` は「親が必要とする引数を、そのまま全部受け取って渡す」ためのおまじない（可変長引数）です。

---

10. 例外処理 try-except

10.1 例外とは

> **例外（れいがい／exception）とは？** プログラム実行中に起きる **エラー（異常事態）**。たとえば「数値に変換できない文字が来た」「探したデータが無かった」など。何もしないと、例外が起きた瞬間にプログラムは **強制終了** します。

10.2 try-except の基本

> **try-except とは？「試して（try）、もしエラーが起きたら（except）こう対処する」という書き方。エラーで止まらず、優雅に対処できます。**
>
> **▼ このコードがやること:** 文字を数値に変換しようとし、できなければエラーメッセージを出す。

コピー

```
# ※説明用の簡易例
try:
    # "abc" を数値に変換しようとする（失敗する）
    value = int("abc")
# ValueError（値の変換エラー）が起きたら
except ValueError:
    # こう対処する（止まらない）
    print("数値ではありません")
```

- `try:` ...「まずこれを試す」。中でエラーが起きるかもしれない処理。
- `except エラー名:` ...「もし指定のエラーが起きたら」。その対処を字下げで書く。
- これにより、`int("abc")` が失敗してもプログラムは止まらず、メッセージを出して続行できます。

10.3 本物のコードで見る try-except

本物のコード（`main_utils.py` の validate_employee_no_logic）：

コピー

```
try:
  # 数値に変換を試みる
  value_int = int(value)
# 変換できなければ（例: 'abc'）
except ValueError:
  return False, 'は自然数で入力して下さい'
```

- ユーザーが従業員番号欄に「abc」など数字でないものを入れたら、`int(value)` が `ValueError` を起こします。
- それを `except` で受け止め、エラーで落ちる代わりに「自然数で入力して下さい」というメッセージを返します。**入力ミスでサーバーが止まらない** ための守りです。

本物のコード（`main_views.py` の Login）— JSON解析の失敗を捕まえる：

コピー

```
try:
  # 送られたデータをPythonの辞書に変換
  data = json.loads(request.body)
  input_number = data.get('employee_no')
# データが壊れた形式なら
except json.JSONDecodeError:
  return JsonResponse({'status': 'error', 'message': 'JSON形式が正しくありません。'}, ...)
```

10.4 Djangoでよく見る「データが無い」例外

本アプリで最頻出なのが、`.get()` でデータを探したが見つからなかった場合の例外です。

本物のコード（`main_views.py` のあちこち）：

コピー

```
try:
  # 該当する人を1人取得
  member_data = member.objects.get(employee_no=login_no)
# その人がいなければ
except member.DoesNotExist:
  return Response({'status': 'error', 'message': 'ユーザー情報が見つかりません。'}, ...)
```

- `member.objects.get(...)` … 条件に合うデータを **1件取得**。
- 見つからないと `member.DoesNotExist`（その人は存在しない）という例外が起きます。
- それを except で受けて、エラーメッセージを返します。「ログインしたが、その後その人が削除された」ようなケースでも安全に対処できます。

10.5 何でも捕まえる except Exception

本物のコード（`asynchronous_views.py` の handle_task）：

コピー

```
# どんなエラーでも捕まえる
except Exception as e:
  ...
  # エラー内容を文字にして記録
  task.result = str(e)
```

- `Exception` … **すべての例外の親**。`except Exception:` は「種類を問わず、何かエラーが起きたら」。
- `as e` … 起きたエラーの中身を変数 `e` に入れる。`str(e)` でエラーメッセージを文字として取り出せます。
- 重い非同期処理（バックアップ等）で「想定外の何が起きても、とにかく記録して止まらない」ための保険です。

> ⚠️ **`except Exception` の乱用は注意**: 何でも捕まえると、本来直すべきバグまで隠してしまうことがあります。本アプリは「どこで何が失敗してもユーザーにエラーを返す」目的で意図的に使っていますが、原因調査のときは捕まえたエラー内容（`str(e)`）をログで確認するのが大切です。

---

11. with 文（ファイルを安全に扱う）

11.1 with とは

> **`with` 文とは？** ファイルなどの「使い終わったら必ず後片付けが必要なもの」を、**自動で後片付けしてくれる** 書き方。「開けたら閉じる」を保証します。

ファイルは、開いたら **必ず閉じる** 必要があります（閉じ忘れるとデータが壊れたり、他の処理が読めなくなる）。`with` を使うと、ブロックを抜けるとき **自動で閉じて** くれます。

11.2 本物のコードで見る with

本物のコード（`main_views.py` の get_logs）：

コピー

```
# ログファイルを開く
with open('web_console.log', 'r') as log_file:
  # 全行をリストで読み込む
  logs = log_file.readlines()
# ← ここでブロックを抜けると、log_file は自動で閉じられる
```

- `open('web_console.log', 'r')` … ファイルを開く。`'r'` は read（読み込み専用）モード。
- `as log_file` … 開いたファイルを `log_file` という名前で扱う。
- `with ...:` の **字下げされた中** がファイルを使う範囲。ここを抜けると **自動でクローズ**。
- `.readlines()` … ファイルの全行をリストにして返す（1行＝1要素）。

> **なぜ with を使う？** `with` を使わないと `log_file.close()` を自分で書く必要があり、途中でエラーが起きると閉じ忘れます。`with` なら **エラーが起きても確実に閉じて** くれるので安全。Pythonでファイルを扱うときの定番です。

11.3 別の with（一時ファイルの作成）

本物のコード（`asynchronous_views.py` の backup）：

コピー

```
with tempfile.NamedTemporaryFile(suffix=".xlsx", delete=False) as temp_file:
  # アップロードされたファイルを小分けに
  for chunk in kosu_file.chunks():
    # 一時ファイルに書き込む
    temp_file.write(chunk)
  # 一時ファイルのパスを覚えておく
  temp_file_path = temp_file.name
```

- `tempfile.NamedTemporaryFile(...)` … 一時的な作業ファイルを作る。
- `for chunk in kosu_file.chunks():` … 大きなファイルを **小さなかたまり（chunk）ずつ** 読み、少しずつ書き込む（メモリを節約）。
- `with` なので、書き終わってブロックを抜けると一時ファイルは自動で閉じられます。

> **`mode='r'` 以外のモード:** `'r'` =読み込み、`'w'` =書き込み（上書き）、`'a'` =追記。`main_views.py` の WebConsoleLogView では `open(LOG_FILE_PATH, 'r', encoding='utf-8')` のように **文字コード（encoding）** も指定して、日本語が文字化けしないようにしています。

---

12. import と相対import

12.1 import とは

> **`import` （インポート）とは？** 他のファイルやライブラリ（部品集）の機能を、自分のファイルで **使えるように読み込む** こと。「道具箱から必要な道具を取り出す」イメージ。

本物のコード（`main_views.py` の冒頭）：

コピー

```
# 日付・時刻を扱う道具箱
import datetime
# OS（ファイル・パス）を扱う道具箱
import os
# JSONを扱う道具箱
import json
# pathlib の中から Path だけを取り出す
from pathlib import Path
# djangoの中から JsonResponse を取り出す
from django.http import JsonResponse
```

2つの書き方があります：

書き方	意味	使うとき
`import datetime`	道具箱ごと読み込む	`datetime.date.today()` のように箱名付きで使う
`from pathlib import Path`	箱から特定の道具だけ取り出す	`Path(...)` のように直接使える

コピー

```
# import datetime した場合 → 箱名を付けて使う
today = datetime.date.today()

# from datetime import date した場合 → 直接使える
today = date.today()
```

12.2 標準ライブラリと外部ライブラリ

- **標準ライブラリ** … Pythonに最初から付属する道具箱（`os`, `json`, `datetime`, `inspect` など）。インストール不要。
- **外部ライブラリ** … 追加でインストールする道具箱（`django`, `rest_framework`, `environ` など）。`pip install` で入れます（`01` 章）。

本物のコード（`main_utils.py`）：

コピー

```
# 標準：オブジェクトを調べる道具
import inspect
# 外部：DRFのページ送り
from rest_framework.pagination import PageNumberPagination
# 外部：Djangoのモデル機能
from django.db import models
```

12.3 相対import（自分のアプリ内のファイルを読む）

> **相対import（そうたいインポート）とは？** ドット `.` を使って「今のファイルから見た位置」で、同じアプリ内の別ファイルを読み込むこと。

書き方	意味
`.module`	ドット1個＝**同じ階層**のファイル
`..module`	ドット2個＝**1つ上の階層**のファイル

本物のコード（`main_views.py`）：

コピー

```
from ..models import member, Business_Time_graph, kosu_division, ...
from .serializers import MemberSerializer, ...
from ..utils.main_utils import CustomPagination, get_all_model_names_in_myapp
```

ファイルの場所を思い出すと（§0.2）、`main_views.py` は `kosu/views/` の中にあります。

- `from ..models import member` … `..`（1つ上＝`kosu/`）の `models.py` から `member` を読む。`views/` から見て `models.py` は1つ上の階層なので `..`。
- `from .serializers import ...` … `.`（同じ階層＝`kosu/views/`）の `serializers.py` から読む。同じフォルダなので `.`。
- `from ..utils.main_utils import ...` … `..`（1つ上＝`kosu/`）の中の `utils` フォルダの `main_utils.py` から読む。

> **なぜ相対import？** フォルダ構成が深いとき、毎回 `from kosu.views.serializers import ...` とフルパスで書くのは長くて壊れやすい。`.` `..` で「今から見た位置」で書けば短く、フォルダを移動しても直しやすい、という利点があります。

本物のコード（`asynchronous_views.py`）— 長い相対import：

コピー

```
from ..tasks import generate_kosu_backup, delete_kosu_data, load_kosu_file, \
                    generate_member_backup, load_member_file, ...
```

> **行末の `\`（バックスラッシュ）**：Pythonでは、1つの文が長くて1行に収まらないとき、行末に `\` を置くと「次の行に続く」という意味になります。たくさんの道具を1つの import 文で読み込むときに使われます。

13. デコレータ @

13.1 デコレータとは

> **デコレータ（decorator）とは？** 関数やクラスの **真上に `@名前` と書いて**、その関数に「追加の機能・性質」を付け加える仕組み。「飾り付け（decorate）」が語源。

イメージは「**ラッピング**」。中身（関数）は同じでも、`@` で包むと「特別な扱い」が追加されます。

13.2 本物のコードで見るデコレータ

本物のコード（`asynchronous_views.py`）：

コピー

```
# ← デコレータ
@api_view(['POST'])
# ← デコレータ
@parser_classes([MultiPartParser, JSONParser, FormParser])
# この関数に上の性質が付く
def backup(request):
  ...
```

- `@api_view(['POST'])` … この関数を「POSTリクエストを受け付けるAPI」にする飾り。これを付けるだけで、Django REST Frameworkが「これはAPIだ」と認識し、POST以外のアクセスを自動で弾いてくれます。
- `@parser_classes([...])` …「受け取るデータの形式（ファイル付き・JSON・フォーム）」を指定する飾り。
- **関数の真上に書く** のがルール。関数1つに複数のデコレータを重ねられます。

13.3 signals.py の @receiver（最重要デコレータ）

本物のコード（`signals.py`）— 本アプリの「自動記録」を支えるデコレータ：

コピー

```
# ← デコレータ
@receiver(post_save, sender=member)
def log_create_update_member_history(sender, instance, created, **kwargs):
  ...
```

- `@receiver(post_save, sender=member)` … この関数を「**member が保存（post_save）されたら自動で呼ばれる** 関数」として登録する飾り。
- これにより、誰かが `member.save()` するたびに、**自分で呼ばなくても** この関数が自動実行され、履歴が記録されます（§18で全解説）。

- `post_save` は「保存の後」、`pre_save` は「保存の前」、`post_delete` は「削除の後」というタイミングを表します。

> **なぜデコレータが便利？**「保存されたら履歴を取る」という処理を、アプリ中の全ての保存箇所に手で書くのは大変＆書き忘れます。`@receiver` で1か所登録しておけば、**どこで保存されても自動で** 履歴が取られます。「仕掛けておけば勝手に動く」のがデコレータの威力です。

13.4 他にもあるデコレータ

> **`@property` など:** デコレータは多種多様で、`@property`（メソッドを属性のように見せる）、`@staticmethod`（インスタンス不要のメソッドにする）などもあります。本アプリでは主に `@api_view`（API化）と `@receiver`（自動実行登録）の2つを押さえれば十分です。

---

14. 実コード全解説①：main_views.py の Login・PrintToLogger・get_logs

ここからは、これまで学んだ文法を総動員して、本物のコードを **1行ずつ** 読み切ります。まずは `kosu/views/main_views.py`。

14.1 import 部分（1〜17行目）

> **▼ このブロックがやること:** このファイルで使う道具（日付・OS・JSON・Django・自作モデルなど）を全部読み込みます。

コピー

```
# 日付・時刻を扱う標準ライブラリ
import datetime
# ファイルパスなどOS機能の標準ライブラリ
import os
# システム関連（標準出力の差し替えに使う）
import sys
# ログ記録の標準ライブラリ
import logging
# .envファイル（環境変数）を読む外部ライブラリ
import environ
# JSONの読み書きの標準ライブラリ
import json
# パス操作の道具 Path を取り出す
from pathlib import Path
# HTMLを描画する関数
from django.shortcuts import render
# JSON形式で応答を返すクラス
from django.http import JsonResponse
# Djangoの基本ビュークラス
from django.views import View
# 設定（settings.py）を読む
from django.conf import settings
# DRFのAPI用ビュークラス
from rest_framework.views import APIView
# DRFの応答クラス
from rest_framework.response import Response
# HTTPステータス番号の定数集
from rest_framework import status
# 自作モデル
from ..models import member, Business_Time_graph, kosu_division, team_member, administrator_data, AsyncTask, History
# 自作シリアライザ
from .serializers import MemberSerializer, AdministratorSerializer, KosuSerializer, DefSerializer, TaskSerializer, HistorySerializer
# 自作の便利部品
from ..utils.main_utils import CustomPagination, get_all_model_names_in_myapp
```

- 上から「標準ライブラリ → 外部ライブラリ（Django/DRF）→ 自作ファイル（相対import）」の順。Pythonの慣習的な並べ方です（§12）。
- `..models`・`.serializers`・`..utils.main_utils` が **相対import**（§12.3）。本アプリ内の別ファイルを読み込んでいます。

> **用語：ロガー（logger）** プログラムの動作記録（ログ）を残す係。`logging` がそれを担当します。**用語：シリアライザ（serializer）** Pythonのデータ（モデル）を、ブラウザに送れるJSON形式に **変換する係**（`07` 章で詳説）。

14.2 ロガーの取得（22〜23行目）

コピー

```
# 専用ロガー取得('views_logger'という名前のカスタムロガーを使用しログメッセージ記録)
views_logger = logging.getLogger('views_logger')
```

- `logging.getLogger('views_logger')` … `'views_logger'` という名前のログ係を取得して、変数 `views_logger` に入れる。
- 以降、`views_logger.info("メッセージ")` でログを記録できます。

14.3 PrintToLogger クラス（27〜38行目）

> ▼ **このブロックがやること:** Pythonの `print()` で出力した内容を、画面に消えてしまわないよう **ログに記録** するための仕掛けです。`print` の行き先（標準出力）を、このクラスにすり替えます。

コピー

```
# クラス定義（§9）
class PrintToLogger:
  # write メソッド（self は自分自身）
  def write(self, message):
    # 受け取った標準出力のメッセージ処理(空白および改行のみのメッセージは無視)
    # 空白・改行だけのメッセージは無視
    if message.strip():
      # 中身があればログに記録
      views_logger.info(message)

  # flush メソッド（必要だが処理は不要）
  def flush(self):
    # 処理不要のため空処理
    # 「何もしない」を表すキーワード
    pass
```

1行ずつ： - `class PrintToLogger:` … クラスの設計図（§9）。 - `def write(self, message):` … `write` というメソッド。`print()` が内部で呼ぶ名前が `write` なので、ここを用意することで print を横取りできます。`self` は自分自身（§9.3）、`message` が print された文字列。 - `if message.strip():` … `.strip()` は文字列の前後の空白・改行を取り除く命令。残った中身があれば（＝空でなければ）真になる（§4.3 truthyの考え方）。 - `views_logger.info(message)` … ログに「情報（info）」として記録。 - `def flush(self):` … `flush` メソッド。標準出力には flush（吐き出し）という機能が必要なので、形だけ用意。 - `pass` … 「**何もしない**」を表すPythonのキーワード。中身を書かないとエラーになるので、空の関数の中身として置きます。

> **用語：** `pass` 「ここは意図的に何もしない」という意味のプレースホルダ。文法上、中身が必要だが書くことがないときに使います。

14.4 標準出力の差し替え（37〜38行目）

コピー

```
# 標準出力をロガーへリダイレクト(sys.stdoutをPrintToLoggerのインスタンスに設定)
sys.stdout = PrintToLogger()
```

- `PrintToLogger()` … クラスから **インスタンスを作る**（§9.1のインスタンス化）。カッコを付けると「たい焼きを1個焼く」イメージ。

- `sys.stdout = ...` … `sys.stdout`（print の出力先）を、作ったインスタンスに **すり替える**。
- これ以降、コード中で `print("xxx")` すると、画面ではなく **ログに残る** ようになります。サーバーは画面が無いので、print をログに飛ばして後から確認できるようにする工夫です。

14.5 get_logs 関数（41〜47行目）

> ▼ **このブロックがやること:** 記録されたログファイルを読み込み、その全行をJSONで返す関数です。管理画面でログを表示するのに使います。

コピー

```
# request を受け取る関数（§7）
def get_logs(request):
  # 'web_console.log' ファイルを読み込み専用モードで開く
  # ファイルを開く（with §11）
  with open('web_console.log', 'r') as log_file:
    # ファイルの内容を行ごとにリストとして格納
    # 全行をリストで読み込む
    logs = log_file.readlines()
  # JSONレスポンスとして、ログを返す
  # 辞書をJSONにして返す（§3.2 §7.3）
  return JsonResponse({'logs': logs})
```

1行ずつ： - `def get_logs(request):` … 関数定義。`request`（ブラウザからの要求）を受け取る。- `with open('web_console.log', 'r') as log_file:` … ログファイルを読み込みモードで開く。`with` なので使い終わると自動で閉じる（§11）。- `logs = log_file.readlines()` … ファイルの全行をリスト（1行＝1要素）にして `logs` に入れる。- `return JsonResponse({'logs': logs})` … `{'logs': logs}` という **辞書**（§3.2）を作り、それをJSON形式の応答にして返す。キー `'logs'`、値がログのリスト。

> ▼ **こう返れば成功:** ブラウザ側にこんなJSONが届きます。 `json {"logs": ["1行目のログ\n", "2行目のログ\n", "..."]}`

14.6 Login クラス（79〜102行目）— 本章の主役

> ▼ **このブロックがやること:** ログイン処理です。送られた従業員番号が人員データに存在するか調べ、あればログイン成功（セッションに記録）、なければエラーを返します。

コピー

```
# ログイン
# APIView を継承（§9.5）
class Login(APIView):
  # POST処理
  # POSTで呼ばれるメソッド
  def post(self, request):
    # まず試す（§10）
    try:
      # 送られてきたデータ確認
      # 受信データをPython辞書に変換
      data = json.loads(request.body)
      # その中の従業員番号を取り出す
      input_number = data.get('employee_no')
    # データが壊れていたら
    except json.JSONDecodeError:
      return JsonResponse({'status': 'error', 'message': 'JSON形式が正しくありません。'},
status=status.HTTP_400_BAD_REQUEST)

    # POSTされた従番が人員データにある場合、セッションに保存&最新工数区分定義取得
    # その番号の人がいるか
    if member.objects.filter(employee_no=input_number).exists():
      # セッションに番号を保存
      request.session['login_No'] = input_number
      # 最新の工数区分定義を取得
      def_Ver = kosu_division.objects.order_by("id").last()

      # 取得した工数区分定義をセッションに保存
      # 定義が1つも無ければ
      if not def_Ver:
        return JsonResponse({'status': 'error', 'message': '利用可能な工数区分がありません。ERROR052'},
status=status.HTTP_400_BAD_REQUEST)
      # 定義があれば
      else:
        # 定義名をセッションに保存
        request.session['input_def'] = def_Ver.kosu_name
        # 成功を返す
        return JsonResponse({'status': 'success'})

    # その番号の人がいなければ
    else:
      return JsonResponse({'status': 'error', 'message': '入力された従業員番号は登録がありません。
ERROR048'}, status=status.HTTP_400_BAD_REQUEST)
```

このコードには、この章で学んだ要素が **ほぼ全部** 詰まっています。1行ずつ：

- `class Login(APIView):` … `Login` クラスを定義。`APIView`（DRFのAPI用親クラス）を **継承**（§9.5）。これでAPIとして動く機能をタダで手に入れます。
- `def post(self, request):` … `post` メソッド。ブラウザが **POSTで** このAPIを叩いたとき呼ばれる。`self`（自分自身 §9.3）と `request`（要求内容）を受け取る。
- `try:` … 以降の処理を試す。失敗するかもしれないから（§10）。
- `data = json.loads(request.body)` … ブラウザから送られた本文（`request.body`）を、`json.loads` でPythonの **辞書** に変換（§3.2）。`loads` は「load string（文字列から読み込む）」。
- `input_number = data.get('employee_no')` … 辞書から `'employee_no'`（従業員番号）を `.get()` で取り出す（§3.2。無ければ None）。

- `except json.JSONDecodeError:` … もしデータが壊れていてJSONとして読めなければ、この行に飛ぶ。
- `return JsonResponse({...}, status=...)` … エラーの辞書をJSONで返す。`status=status.HTTP_400_BAD_REQUEST` は「400番（リクエストが不正）」というHTTPの番号（`07` 章）。
- `if member.objects.filter(employee_no=input_number).exists():` … `member`（人員モデル §17）から、従業員番号が一致する人を探し、`.exists()` で「いるか（True/False）」を判定。これが条件分岐（§4）の中心。
- `request.session['login_No'] = input_number` … セッション（ログイン状態を覚える仕組み）に従業員番号を保存。`request.session` は辞書のように使えます。
- `def_Ver = kosu_division.objects.order_by("id").last()` … 工数区分定義を **id順に並べて最後の1件**（＝最新版）を取得。
- `if not def_Ver:` … `def_Ver` が None（1件も無い）なら真（§4.3 falsy）。定義が無ければエラーを返す。
- `else:` … 定義があれば、その名前 `def_Ver.kosu_name` をセッションに保存し、`{'status': 'success'}` を返す。
- 最後の `else:` … 最初の if（人がいるか）が偽だったとき。「登録がありません」とエラーを返す。

> **用語：セッション（session）**「この人は今ログイン中」という状態を、サーバーが一時的に覚えておく仕組み。`request.session['login_No']` に従業員番号を入れておくと、次回以降のアクセスで「誰がログイン中か」が分かります（`07` 章で詳説）。
>
> **用語：`.filter()` と `.get()` と `.exists()`（Djangoのデータ操作）** - `.filter(条件)` … 条件に合うデータを **複数** 探す（0件以上の集まり）。- `.get(条件)` … 条件に合うデータを **1件だけ** 取る（無いと例外 §10.4）。- `.exists()` …「該当データがあるか」を True/False で返す。- `.order_by("id").last()` … id で並べ替えて最後の1件。`.first()` なら最初の1件。これらはDjangoの機能で、`07` 章で本格的に学びます。ここでは「データベースを操作する命令なんだな」と分かれば十分です。

14.7 TeamMenu の班員チェック（340〜378行目）— for・内包表記・f文字列の総合

§5〜§8で部分的に見た TeamMenu のループを、まとめて読みます。

> **▼ このブロックがやること:** 班のメンバーそれぞれについて、過去7日間に工数の未入力がないかを調べ、未入力があれば「○○氏の工数未入力があります。」というメッセージを作ります。

コピー

```
# 班員フォローメッセージ作成
# 空のリスト（§3.1）
follow_message_list = []
# 班員データあるか確認
team_filter = team_member.objects.filter(employee_no5=login_no)
# 三項演算子（§8.1）
team_get = team_filter.first() if team_filter.exists() else None

# 班員フォロー有効の場合、班員リスト作成
# and で2条件（§4.3）
if team_filter.exists() and team_get.follow:
  # 15人分の番号をリストに
  member_numbers = [
    team_get.member1, team_get.member2, team_get.member3,
    team_get.member4, team_get.member5, team_get.member6,
    team_get.member7, team_get.member8, team_get.member9,
    team_get.member10, team_get.member11, team_get.member12,
    team_get.member13, team_get.member14, team_get.member15
  ]
  # 内包表記（§6.2）
  valid_member_numbers = [
    num for num in member_numbers if num is not None and num != ''
  ]

  # 過去7日分の日付リスト作成
  # 今日の日付
  today = datetime.date.today()
  # 内包表記（§6.3）
  day_list = [today - datetime.timedelta(days=d) for d in range(1, 8)]

  # 班員ごとに工数未入力確認
  # 班員を1人ずつ（§5）
  for m in valid_member_numbers:
    follow_message = ''
    # その人がいるか
    if member.objects.filter(employee_no=m).exists():
      # 名前を取得
      member_name = member.objects.get(employee_no=m).name
      # 7日を1日ずつ（入れ子ループ）
      for d in day_list:
        if Business_Time_graph.objects.filter(employee_no3=m, work_day2=d).exists():
          kosu_get = Business_Time_graph.objects.get(employee_no3=m, work_day2=d)
          # 判定がOKでなければ
          if not kosu_get.judgement:
            # f文字列（§8.2）
            follow_message = f'{member_name}氏の工数未入力があります。'
            # リストに追加（§3.1）
            follow_message_list.append(follow_message)
```

```
            # このループを抜ける（§5.4）
            break
        # その日のデータ自体が無ければ
        else:
            follow_message = f'{member_name}氏の工数未入力があります。'
            follow_message_list.append(follow_message)
            break
```

ポイントの確認：- `follow_message_list = []` … 空リストを用意して、後で `.append()` で詰める定番（§3.1）。- `team_get = ... if ... else None` … 三項演算子（§8.1）。- `member_numbers = [...]` … 15人分の番号を1つのリストに。- `valid_member_numbers = [num for num in ... if ...]` … 条件付き内包表記（§6.2）で空でない番号だけ抽出。- `day_list = [... for d in range(1, 8)]` … 内包表記（§6.3）で過去7日。- 二重 for ループ（§5.2）で、各班員×各日を調べる。- `f'{member_name}氏の...'` … f文字列（§8.2）で名前を埋め込む。- `break` （§5.4）で、1件見つけたら無駄を省いて抜ける。

1つのコードブロックの中に、リスト・辞書アクセス・三項演算子・内包表記・二重ループ・f文字列・break が全部詰まっています。これが読めれば、本アプリのバックエンドの大半が読めます。

---

15. 実コード全解説②：main_utils.py の検証関数

`kosu/utils/main_utils.py` 全体を読みます。短いファイルですが、関数・クラス・例外・継承の良い見本です。

15.1 import（1〜6行目）

コピー

```
# オブジェクトの中身を調べる標準ライブラリ
import inspect
# DRFのページ送り機能
from rest_framework.pagination import PageNumberPagination
# DRFの応答クラス
from rest_framework.response import Response
# Djangoのモデル基本機能
from django.db import models
# 自作モデルを myapp_models という別名で
from kosu import models as myapp_models
# 設定モデルを相対importで
from ..models import administrator_data
```

- `from kosu import models as myapp_models` … `as` は **別名（あだ名）を付ける** キーワード。`kosu.models` を `myapp_models` という短い名前で呼べるようにする。後で「自作モデル全部」を調べるときに使います（§15.4）。

15.2 CustomPagination クラス（11〜25行目）

> ▼ **このブロックがやること:** 一覧表示の「ページ送り（1ページ20件など）」の部品です。1ページの件数を、管理者設定から動的に読み込みます。

コピー

```
# ページネーションクラス
# 既製のページ送りを継承（§9.5）
class CustomPagination(PageNumberPagination):
  # デフォルトの設定値              # クラス変数（初期値20件）
  page_size = 20

  # 作られるとき自動実行（§9.2）
  def __init__(self):
    # administrator_data から動的にページサイズを設定
    # 最新の設定を取得
    last_record = administrator_data.objects.order_by("id").last()
    # 設定があれば（§4.4）
    if last_record is not None:
      # 1ページの件数を設定値に上書き
      self.page_size = last_record.menu_row

  # ページ送り済みの応答を作るメソッド
  def get_paginated_response(self, data):
    # 辞書を応答にして返す（§3.2）
    return Response({
      # 合計件数
      'count': self.page.paginator.count,
      # ページサイズ
      'page_size': self.page_size,
      # 現在のページのデータ
      'results': data,
    })
```

1行ずつ： - `class CustomPagination(PageNumberPagination):` … DRFの `PageNumberPagination`（番号でページ送りする機能）を **継承**。基本機能はタダで手に入れ、一部だけ自分流に変えます。 - `page_size = 20` … **クラス変数**（クラス全体で共有する初期値）。「1ページ20件」がデフォルト。 - `def __init__(self):` … 初期化メソッド（§9.2）。このページ送り部品が作られた瞬間に実行。 - `last_record = administrator_data.objects.order_by("id").last()` … 管理設定（administrator_data）の最新1件を取得。 - `if last_record is not None:` … 設定が存在すれば（§4.4の `is not None`）。 - `self.page_size = last_record.menu_row` … `self.page_size`（この部品のページ件数）を、設定の `menu_row`（一覧表示件数）で上書き。`self.` が付くことで「このインスタンスの page_size」を変更（§9.3）。 - `def get_paginated_response(self, data):` … ページ送り済みデータを応答にするメソッド。 - `return Response({...})` …「合計件数・ページサイズ・データ本体」をまとめた辞書を応答として返す。

なぜ設定から件数を読む？「一覧に何件表示するか」を管理者が設定画面で変えられるようにするためです。コードに `20` と固定で書くと変更にコード修正が必要ですが、設定から読めば **画面から変えられる** ようになります。

15.3 validate_employee_no_logic 関数（29〜50行目）

▼ このブロックがやること: 入力された従業員番号が正しいか（自然数か・実在するか）を検証する関数です。「OKか（True/False）」と「理由メッセージ」をペアで返します。

コピー

```
# 2つの引数（§7）
def validate_employee_no_logic(value, member_model):
  # 1. 空欄チェック: 値が存在しない、または空文字列の場合は空欄を許可する
  # 値が空なら（§4.3 falsy）
  if not value:
    # 「OK・理由なし」を返す（§7.4）
    return True, None

  # 2. 整数変換チェック
  # 試す（§10）
  try:
    # 数値に変換
    value_int = int(value)
  # 変換できなければ
  except ValueError:
    # intに変換できなかった場合（例: 'abc'などの文字列）
    # 「NG・理由」を返す
    return False, 'は自然数で入力して下さい'

  # 3. 自然数チェック（1以上の整数）
  # 0以下なら（§4.2）
  if value_int <= 0:
    return False, 'は自然数で入力して下さい'

  # 4. 従業員番号の存在チェック
  # 試す
  try:
    # その番号の人を取得
    member_model.objects.get(employee_no=value_int)
    # いれば「OK・番号」を返す
    return True, value_int
  # いなければ
  except member_model.DoesNotExist:
    return False, 'に入力された従業員番号の人員は存在しません'
```

1行ずつ： - `def validate_employee_no_logic(value, member_model):` … 関数定義。`value`（検証する値）と `member_model`（人員モデル）を受け取る。- `if not value:` … 値が空（空文字や None）なら真（§4.3）。- `return True, None` … 空欄は許可なので「成功・理由なし」の **2値** を返す（§7.4）。- `try: value_int = int(value)` … 数値への変換を試みる（§10）。- `except ValueError: return False, '...'` … 「abc」など数字でない値なら変換に失敗 → 「失敗・理由」を返す。- `if value_int <= 0:` … 変換できても0以下（マイナスや0）なら自然数でない → 失敗を返す。- `try: member_model.objects.get(employee_no=value_int)` … その番号の人を1人取得（§14.6の `.get()`）。- `return True, value_int` … 取得できたら「成功・その番号」を返す。- `except member_model.DoesNotExist: return False, '...'` … その人がいなければ（§10.4の例外）「失敗・理由」を返す。

> **この関数の設計の美しさ:** すべての枝（空欄・変換失敗・0以下・不存在・成功）が必ず `return 成否, 理由` の **同じ形** で返ります。呼び出す側は `ok, msg = validate_employee_no_logic(...)` で受けて、`if not ok:` でエラー処理ができます。「成否と理由をペアで返す」のは、検証関数の王道パターンです。

15.4 get_all_model_names_in_myapp 関数（56〜68行目）

> **▼ このブロックがやること:** 本アプリの全モデル（データベースのテーブル）の名前を、自動で一覧にして返す関数です。手で書かずプログラムが見つけ出します。

コピー

```
# 引数なしの関数（§7.3）
def get_all_model_names_in_myapp():
  # 空のリスト（§3.1）
  model_names = []

  # myapp_modelsモジュールのメンバーを全て取得
  # models.py の中身を1つずつ
  for name, obj in inspect.getmembers(myapp_models):
    # それがクラスで（§4.3 and）
    if (inspect.isclass(obj) and
      # models.Model を継承していて
      issubclass(obj, models.Model) and
      # 抽象クラスでなく
      not obj._meta.abstract and
      # プロキシでなく
      not obj._meta.proxy and
      # この自作モジュール由来である
      obj.__module__ == myapp_models.__name__):
      # 条件を満たせば名前をリストに追加
      model_names.append(name)

  # 集めた名前リストを返す
  return model_names
```

1行ずつ： - `def get_all_model_names_in_myapp():` … 引数なしの関数。- `model_names = []` … 結果を入れる空リスト。- `for name, obj in inspect.getmembers(myapp_models):` … `inspect.getmembers` は「あるモジュールの中身（名前と実体のペア）」を全部返す道具。それを `name`（名前）と `obj`（実体）に分けて1個ずつ取り出す。**for で2つの変数に同時に受け取る** 書き方（タプルの分解）。- `if (... and ... and ...):` … 複数条件を `and` でつないで、すべて満たすか判定（§4.3）。カッコで囲むと複数行に分けて書けます。- `inspect.isclass(obj)` … それがクラスか。- `issubclass(obj, models.Model)` … `models.Model` を継承したクラスか（＝Djangoのモデルか §9.5）。- `not obj._meta.abstract` … 抽象クラス（直接使わない雛形）でないか（`not` で否定 §4.3）。- `not obj._meta.proxy` … プロキシ（別名）でないか。- `obj.__module__ == myapp_models.__name__` … この自作モデルファイル由来か（importしてきた他所のモデルを除外）。- `model_names.append(name)` … 全条件OKなら、その名前をリストに追加。- `return model_names` … 集めた名前リストを返す。

> **なぜこんな自動化を？** 履歴画面の絞り込みで「テーブル名」の選択肢を出すとき（`main_views.py` の AdministratorHistoryList で `get_all_model_names_in_myapp()` を使用）、モデルを追加するたびに選択肢を手で増やすのは面倒＆書き忘れます。この関数を使えば、**モデルを増やしても選択肢が自動で増える。**
> `inspect` で自分自身のコードを調べる、少し高度だが実用的なテクニックです。
>
> **用語：抽象クラス（abstract）/ プロキシ（proxy）** どちらもDjangoの特殊なモデルで、実際のテーブルを持たない雛形や別名です。履歴の対象は「本物のテーブル」だけにしたいので、これらを `not` で除外しています。

16. 実コード全解説③：asynchronous_views.py の validate_dates

`kosu/views/asynchronous_views.py` から、日付の検証関数 `validate_dates` を中心に読みます（バックアップ処理の全体は `07` 章で扱います）。

16.1 validate_dates 関数（200〜215行目）

> ▼ **このブロックがやること:** バックアップや削除で指定された「開始日・終了日」が正しいかを検証します。空でないか・正しい日付の形か・昨日以前か・開始が終了より前か、を順にチェックします。

コピー

```
# 日付バリデーション関数
# 2つの日付を受け取る
def validate_dates(start_day, end_day):
  # 今日を 'YYYY-MM-DD' の文字に
  today_str = datetime.date.today().strftime('%Y-%m-%d')
  # どちらか空なら（§4.3 or）
  if not start_day or not end_day:
    return JsonResponse({'status': 'error', 'message': '日付を指定してください。'}, status=status.HTTP_400_BAD_REQUEST)

  # 日付変換を試す（§10）
  try:
    # 文字→日付オブジェクト
    end_date_obj = datetime.date.fromisoformat(end_day)
    today_date_obj = datetime.date.fromisoformat(today_str)
    start_date_obj = datetime.date.fromisoformat(start_day)
  # 形式が不正なら
  except ValueError:
    return JsonResponse({'status': 'error', 'message': '日付の形式が不正です。'}, status=status.HTTP_400_BAD_REQUEST)
  # 終了日が今日以降なら（§4.2）
  if end_date_obj >= today_date_obj:
    return JsonResponse({'status': 'error', 'message': '昨日の日付までしか指定できません。'}, status=status.HTTP_400_BAD_REQUEST)
  # 開始日が終了日より後なら
  if start_date_obj > end_date_obj:
    return JsonResponse({'status': 'error', 'message': '開始日が終了日を超えています。'}, status=status.HTTP_400_BAD_REQUEST)
  # 全部OKなら None（問題なし）を返す
  return None
```

1行ずつ： - `def validate_dates(start_day, end_day):` … 開始日と終了日（どちらも文字列）を受け取る関数。- `today_str = datetime.date.today().strftime('%Y-%m-%d')` … `datetime.date.today()` で今日の日付を取得し、`.strftime('%Y-%m-%d')` で「2026-06-01」のような **文字列** に整形。`%Y` =年、`%m` =月、`%d` =日。- `if not start_day or not end_day:` … 開始日か終了日のどちらかが空なら（§4.3の falsy と or）。- `return JsonResponse({...}, status=...)` … エラーをJSONで返す。- `try:` … 文字列を日付オブジェクトに変換するのを試す。- `datetime.date.fromisoformat(end_day)` … 「2026-06-01」のような文字列を、計算できる **日付オブジェクト** に変換。`fromisoformat` は「ISO形式（YYYY-MM-DD）の文字から作る」。- `except ValueError:` … 「2026/13/99」のような不正な文字なら変換に失敗 →「形式が不正」エラーを返す。- `if end_date_obj >= today_date_obj:` … 日付オブジェクト同士は `>=` で **大小比較** できる。終了日が今日以降ならエラー（「昨日までしか指定できない」＝確定したデータだけ扱う）。- `if start_date_obj > end_date_obj:` … 開始日が終了日より後ならエラー（期間が逆転している）。- `return None` … すべての検査を通過したら `None` を返す。

この関数の `None` の意味: この関数は「**問題があればエラー応答を、問題なければ None を返す**」という設計です。呼び出す側（backup 関数）は： `python error_response = validate_dates(start_day, end_day) if error_response: # None でなければ（＝エラーがあれば） return error_response # そのエラーをそのまま返す` `if error_response:` は「error_response が None（falsy）でなければ」という意味（§4.3）。None なら次へ進み、エラー応答なら即座にそれを返す——「問題なければ None」という慣習をうまく使っています。

用語：日付オブジェクト `datetime.date` 型の値。ただの文字「2026-06-01」と違い、引き算（§6.3 の timedelta）や大小比較ができます。文字のままでは正しく比較できないので、`fromisoformat` で日付オブジェクトに変換してから比較しています。

16.2 デコレータ付きの backup 関数の頭（21〜27行目）

§13で見たデコレータと、§3.3のタプル、§10の例外、§11のwithが backup 関数に集約されています。冒頭だけ再確認します。

コピー

```
# API化デコレータ（§13.2）
@api_view(['POST'])
# データ形式指定デコレータ
@parser_classes([MultiPartParser, JSONParser, FormParser])
def backup(request):
  # タスクID生成
  # 一意なIDを文字列で生成
  task_id = str(uuid.uuid4())
  # 処理中レコードを作成
  AsyncTask.objects.create(task_id=task_id, status='pending')

  # 開始日を取得（§3.2 .get）
  start_day = request.data.get('start_day')
  # 終了日を取得
  end_day = request.data.get('end_day')
```

- `@api_view(['POST'])` … POST専用APIにする飾り（§13.2）。
- `str(uuid.uuid4())` … `uuid.uuid4()` で世界に1つの一意なID（重複しない長い識別子）を生成し、`str(...)` で文字列に変換。タスクを区別する番号札です。
- `AsyncTask.objects.create(...)` …「処理中（pending）」のタスク記録を作成（§17の AsyncTask モデル）。
- `request.data.get('start_day')` … 送られたデータから開始日を取り出す（無ければ None）。

> backup 関数は、URLの名前（ `url_name` ）に応じて「どのバックアップ処理を、どんな引数で実行するか」を `if/elif` で振り分け、別スレッドで実行します。その全体像と非同期処理（ `threading` ）は `07` 章で扱います。ここでは「デコレータ・タプル・例外・with という、本章の文法が実戦投入されている」ことを確認できれば十分です。

---

17. 実コード全解説④：models.py の member クラスの形

`kosu/models.py` の `member` クラスを読みます。これは **データベースのテーブルの設計図**（§9のクラスの一種）で、本アプリの中心的なデータです。

17.1 クラス定義と選択肢リスト（6〜21行目）

> **▼ このブロックがやること:** 「人員（社員）」を表すテーブルの設計図です。冒頭で「ショップ」欄の選択肢候補をタプルのリストで定義します。

コピー

```
# models.Model を継承（§9.5）
class member(models.Model):
  # ショップの選択肢（タプルのリスト §3.3）
  shop_list = [
    # ('保存値', '表示値')
    ('P', 'P'),
    ('R', 'R'),
    ('W1', 'W1'),
    ('W2', 'W2'),
    ('T1', 'T1'),
    ('T2', 'T2'),
    ('A1', 'A1'),
    ('A2', 'A2'),
    ('J', 'J'),
    ('その他', 'その他'),
    ('組長以上(P,R,T,その他)', '組長以上(P,R,T,その他)'),
    ('組長以上(W,A)', '組長以上(W,A)'),
    ('異動・退社', '異動・退社'),
  ]
```

- `class member(models.Model):` … `member` クラスを定義し、`models.Model` を継承。これで「データベースのテーブルになれる」機能を得ます（§9.5）。

- `shop_list = [...]` … クラス変数として選択肢を定義。`[ ]` のリスト（§3.1）の中に `('P', 'P')` などの **タプル**（§3.3）が並ぶ構造。各タプルは `('データベースに保存する値', '画面に表示する値')` のペアです。

17.2 フィールド定義（23〜62行目）

> ▼ **このブロックがやること:** この人員テーブルが持つ「列（カラム）」を1つずつ定義します。従業員番号・氏名・ショップ・権限・各種休憩時間など。

コピー

```
# 整数の列
employee_no = models.IntegerField('従業員番号')
# 短い文字の列（最大100字）
name = models.CharField('氏名', max_length=100)
# 選択肢付き文字
shop = models.CharField('ショップ', choices = shop_list, max_length=15)
# 真偽値の列（True/False）
authority = models.BooleanField('権限')
# 真偽値の列
administrator = models.BooleanField('管理者')
# 休憩時間（文字9字）
break_time1 = models.CharField('1直昼休憩時間', max_length=9)
break_time1_over1 = models.CharField('1直残業休憩時間1', max_length=9)
...
pop_up1 = models.CharField('ポップアップ1', max_length=255, null=True, blank=True)
...
break_check = models.BooleanField('休憩エラー有効チェック', null=True)
```

各行の意味： - `フィールド名 = models.〇〇Field('表示名', オプション)` … テーブルの1列を定義する形。 - `models.IntegerField(...)` … **整数** を入れる列（従業員番号など）。 - `models.CharField('...', max_length=100)` … **短い文字列** を入れる列。`max_length` は最大文字数（必須）。 - `models.BooleanField('...')` … **真偽値**（True/False §2.2）を入れる列。権限の有無など「はい/いいえ」。 - `choices=shop_list` … その列の入力を、§17.1で定義した **選択肢に限定**。プルダウンの候補になります。 - `null=True` … データベースで「**値なし（None）を許可**」する設定。 - `blank=True` … フォームで「**空欄を許可**」する設定。 - `'従業員番号'` のような第1引数 … 管理画面などに表示される **日本語の名前（ラベル）**。

> **用語：フィールド（field）** テーブルの「列（カラム）」のこと。Excelの表でいう「列見出し」。member テーブルなら「従業員番号」「氏名」「ショップ」などが各フィールド。**用語：モデル（model）** Djangoで「データベースのテーブルを表すクラス」のこと。member クラス＝member テーブルの設計図、という対応です（ 07 章で詳説）。
>
> **なぜ `null=True` と `blank=True` の両方？** `null` は **データベース** のレベル、`blank` は **入力フォーム** のレベルの「空を許す」設定です。両方付けると「画面で空欄でOK、かつデータベースにも空（None）で保存できる」状態になります。ポップアップ欄など「任意入力」の項目に付いています。

17.3 `__str__` メソッド（64〜65行目）

コピー

```
  # 文字列表現メソッド（§9.4）
  def __str__(self):
    # この人を表示するときは氏名を出す
    return self.name
```

- `def __str__(self):` … このインスタンスを文字として表示するときに呼ばれる特別メソッド（§9.4）。
- `return self.name` …「この member の `name`（氏名）」を返す。`self.name` の `self` は「この人自身」（§9.3）。
- これにより、Django管理画面や履歴で member を表示すると、idの数字でなく **氏名** が出て分かりやすくなります。

17.4 他モデルの `__str__` も見ておく

`models.py` の他のモデルの `__str__` には、§8.2のf文字列や§9.4の文字列連結が使われています。

コピー

```
# Business_Time_graph （文字列連結 §9.4）
def __str__(self):
  return str(self.id) + '__' + str(self.work_day2) + ':' + str(self.employee_no3)

# AsyncTask （f文字列 §8.2）
def __str__(self):
  return f'{self.created_at} on {self.status} (TaskID: {self.task_id})'

# History （f文字列 §8.2）
def __str__(self):
  return f'{self.operation} on {self.table_name} (ID: {self.record_id})'
```

> どれも「人間が見て分かりやすい1行の表示名」を作っています。`+` で連結する書き方（数値は `str()` で文字化）と、f文字列 `f'...{値}...'` の書き方、両方が実戦で使われています。

17.5 save をオーバーライドする（継承＋super の実例）

`models.py` の History モデルには、§9.5で触れた `super()` の実例があります。

> **▼ このブロックがやること:** 履歴を保存するたびに、保存後の総件数を数え、上限（50万件）を超えていたら古いものから削除します。

コピー

```
class History(models.Model):
  # 上限件数（クラス変数）
  MAX_RECORDS = 500000
  ...
  # 親の save を上書き（オーバーライド）
  def save(self, *args, **kwargs):
    # まず親（Model）の保存を実行（§9.5）
    super().save(*args, **kwargs)
    # 現在の総件数を数える
    current_count = self.__class__.objects.count()

    # レコード数が許容数以上の場合の処理
    # 上限を超えたら
    if current_count > self.MAX_RECORDS:
      # 超過レコード数分のレコード取得し削除
      # 何件オーバーか
      excess_count = current_count - self.MAX_RECORDS
      # 古い順に超過分
      oldest_records = self.__class__.objects.order_by('timestamp')[:excess_count]
      # 1件ずつ（§5）
      for record in oldest_records:
        # 削除
        record.delete()
```

1行ずつのポイント： - `def save(self, *args, **kwargs):` … 親（models.Model）が持つ `save`（保存）メソッドを **上書き（オーバーライド）**。`*args, **kwargs` は「親が必要とする引数を全部受け取る」おまじない（§9.5）。- `super().save(*args, **kwargs)` … `super()` で **親の save** を呼び、まず普通に保存する（§9.5）。これを忘れると保存されません。- `current_count = self.__class__.objects.count()` … `self.__class__` は「このクラス自身（History）」。`.count()` で総件数を数える。- `if current_count > self.MAX_RECORDS:` … 上限（50万）を超えたか。- `excess_count = current_count - self.MAX_RECORDS` … 超過した件数。- `[:excess_count]` … リストの **スライス**。「先頭から excess_count 個まで」を取り出す（古い順なので、古いものを超過分だけ）。- `for record in oldest_records: record.delete()` … 取り出した古いレコードを1件ずつ削除。

> **用語：オーバーライド（override）** 継承した親のメソッドを、子クラスで **同じ名前で書き直して上書き** すること。「親のやり方をベースに、自分流に作り変える」。ここでは「保存」に「古いデータの自動削除」を足しています。**用語：スライス（slice）** `リスト[開始:終了]` でリストの一部を切り出す書き方。`[:5]` は「先頭から5個」、`[2:]` は「2番目から最後まで」。

18. 実コード全解説⑤：signals.py の差分計算

最後に `kosu/signals.py` を読みます。`@receiver`（§13.3）で「データが変わったら自動で履歴を取る」仕掛けの中核です。少し難しいですが、これまでの文法の集大成です。

18.1 import と初期化（1〜11行目）

コピー

```
# スレッドごとの保管庫
from threading import local
# 保存・削除の合図
from django.db.models.signals import pre_save, post_save, post_delete
# 合図を受け取る登録用デコレータ
from django.dispatch import receiver
# 自作モデル（相対import §12）
from .models import History, member, Business_Time_graph, ...
# 現在のリクエスト取得
from .middleware.clear_session_middleware import get_current_request
from django.db import models

# スレッドローカル変数初期化
# スレッドごとの一時保管庫を作る
_thread_locals = local()
```

- `from django.db.models.signals import pre_save, post_save, post_delete` … 「保存の前・保存の後・削除の後」という **タイミングの合図（シグナル）** を読み込む。
- `from django.dispatch import receiver` … `@receiver` デコレータ（§13.3）を読み込む。
- `_thread_locals = local()` … **スレッドローカル**（同時に動く複数の処理が、互いに干渉せず使える保管庫）を作る。「更新前の値」を一時的にしまっておく場所。

> **用語：シグナル（signal）** Djangoで「○○が起きたよ」という **合図**。`post_save`（保存された後）の合図に関数を登録しておくと、保存のたびにその関数が自動で呼ばれます（§13.3）。**用語：スレッド（thread）** 同時並行で走る処理の流れ。複数人が同時にアクセスすると、サーバーは複数のスレッドで処理します。「スレッドローカル」は、その各流れが互いの値を踏まないよう分けて持つ保管庫です。

18.2 キャッシュの保存・取得（14〜27行目）

> **▼ このブロックがやること:** データを更新する直前に「更新前の値」を一時保管し、後で「変更前と後」を比べられるようにします。

コピー

```
# 更新前の値をスレッドローカルキャッシュに保存
# instance（保存される対象）を受け取る
def set_instance_cache(instance):
  # そのオブジェクトのクラス（種類）を取得
  model = type(instance)
  # 試す（§10）
  try:
    # 更新前の値取得→スレッドローカルキャッシュに保存
    # DBの現在値を保管
    _thread_locals.instance_cache = model.objects.get(pk=instance.pk)
  # まだDBに無ければ（新規作成）
  except model.DoesNotExist:
    # レコードなしの場合はNone取得
    # Noneを保管
    _thread_locals.instance_cache = None


# スレッドローカルキャッシュから更新前の値取得
def get_instance_cache():
  # 保管した値を返す（無ければNone）
  return getattr(_thread_locals, 'instance_cache', None)
```

- `model = type(instance)` … `type(...)` は「その値の **型（クラス）**」を返す。保存対象が member なら member クラスが返る。
- `_thread_locals.instance_cache = model.objects.get(pk=instance.pk)` … `pk`（主キー＝id）で **データベースの現在値** を取得し、保管庫に入れる。これが「更新前の姿」。
- `except model.DoesNotExist:` … まだ保存前（新規）なら、更新前の値は無いので None を保管。
- `getattr(_thread_locals, 'instance_cache', None)` … `getattr(対象, '属性名', 初期値)` は「対象からその属性を取る。無ければ初期値」。安全に取り出す関数です。

> **用語： pk （ピーケー／primary key：主キー）** 各レコードを一意に区別する番号。通常は `id` 。`instance.pk` で「このレコードのid」が分かります。**用語：キャッシュ（cache）** 「後で使うために一時的に取っておくこと」。ここでは「更新前の値」をキャッシュ（保管）しておき、更新後と比べます。

18.3 差分計算 get_changes（32〜106行目）

> **▼ このブロックがやること:** 「更新前」と「更新後」を全フィールドで比べ、変わった項目だけを `{項目名: {old: 前, new: 後}}` という辞書にまとめます。これが履歴の中身になります。

長いので、前半（変更を検出する部分）から読みます。

コピー

```
# 保存対象と「新規か(created)」を受け取る
def get_changes(instance, created):
  # 変更点を入れる空の辞書（§3.2）
  changes = {}

  # 1. キャッシュを取得
  # §18.2で保管した更新前の値
  old_instance = get_instance_cache()

  # 【修正】キャッシュが存在しても、現在のインスタンスと型やPKが違う場合は、誤った比較を避けるためNoneにする
  if old_instance and (not isinstance(old_instance, type(instance)) or old_instance.pk != instance.pk):
    # 別物なら比較対象を無効化
    old_instance = None

  # モデルの全フィールド処理
  # 全フィールドを1つずつ（§5）
  for field in instance._meta.fields:
    # フィールド名（例 'name'）
    field_name = field.name

    # 値取得
    # 新しい値を取得（getattrで動的に）
    new_value = getattr(instance, field_name)

    # 新規作成時、全て変更として処理
    # 新規作成なら
    if created:
      # 古い値は無い
      old_value = None
      # 全項目を「変更あり」とする
      is_changed = True
    # 更新時、差分取得
    # 更新なら
    else:
      # 更新前の値があれば
      if old_instance:
        # 古い値を取得
        old_value = getattr(old_instance, field_name)
        # 古い≠新しい なら変更あり（§4.2）
        is_changed = (old_value != new_value)
      else:
        # 【修正】pre_saveでのキャッシュ漏れやスレッドの混同対策としてDBから直接取得を試みる
        try:
          # DBから直接取得
          old_db_instance = type(instance).objects.get(pk=instance.pk)
          old_value = getattr(old_db_instance, field_name)
          is_changed = (old_value != new_value)
          # 以降のためにキャッシュ更新
          old_instance = old_db_instance
        except type(instance).DoesNotExist:
          # 取れなければこの項目は飛ばす（§18.4）
          continue
```

要点： - `changes = {}` … 変更点をためる **空の辞書**（§3.2）。最後にこれを返す。- `old_instance = get_instance_cache()` … 更新前の値を取り出す（§18.2）。- `if old_instance and (not isinstance(...) or old_instance.pk != instance.pk):` … `isinstance(A, 型)` は「A がその型か」を判定。型やidが食い違う（別物）なら、誤比較を避けて `old_instance = None`。- `for field in instance._meta.fields:` … モデルの全フィールド（列）を1つずつ処理（§5）。`_meta.fields` がフィールド一覧。- `new_value = getattr(instance, field_name)` … `getattr(対象, '名前')` で、名前を **文字列で指定して** 値を取り出す。フィールド名が変数なので、`instance.name` と直接書けないため getattr を使う。- `if`

`created:` … 新規作成なら、古い値なし・全項目を変更扱い。- `else:` 以降 … 更新なら、古い値と新しい値を `!=`（等しくない §4.2）で比べ、違えば `is_changed = True`。- `continue` …「この項目は飛ばして次のループへ」（後述§18.4）。

続く後半（変更があった項目の値を、JSONで保存できる形に整える部分）：

コピー

```
    # 変更or新規作成
    if is_changed:

      # JSON化できないリレーションフィールド処理
      # 他テーブルへの参照なら
      if field.is_relation and field.many_to_one:
        # 関連ID（三項演算子 §8.1）
        old_json_safe_value = getattr(old_instance, field.attname) if old_instance else None
        new_json_safe_value = getattr(instance, field.attname)

      # その他オブジェクト処理
      # 基本型でなければ
      elif not isinstance(new_value, (str, int, float, bool, type(None))):
        # モデルそのものなら
        if isinstance(new_value, models.Model):
          # 辞書化
          old_json_safe_value = {'id': old_value.pk, 'str': str(old_value)} if old_value else None
          new_json_safe_value = {'id': new_value.pk, 'str': str(new_value)}
        # 日付なら
        elif isinstance(new_value, (models.DateField, models.DateTimeField)):
          # 文字列化
          old_json_safe_value = old_value.isoformat() if old_value else None
          new_json_safe_value = new_value.isoformat()
        # その他は文字列化
        else:
          old_json_safe_value = str(old_value) if old_value else None
          new_json_safe_value = str(new_value)

      # JSON化可能な基本データ型の処理
      # 数値・文字・真偽値など
      else:
        old_json_safe_value = old_value
        new_json_safe_value = new_value

      # 6. changes辞書に記録
      # 新規なら
      if created:
        # 新しい値だけ記録
        changes[field_name] = new_json_safe_value
      # 更新なら
      else:
        # 前後を記録
        changes[field_name] = {'old': old_json_safe_value, 'new': new_json_safe_value}

  # 変更辞書を返す
  return changes
```

要点： - `if is_changed:` … 変更があった項目だけ処理。- `if ... elif ... else:` の三段（§4.1）で、値の種類（参照・モデル・日付・基本型）ごとに **JSONで保存できる形** に変換。- `isinstance(new_value, (str,`

`int, float, bool, type(None)))` … 値が基本型のどれかか。複数の型をタプルで渡してまとめて判定。- 日付は `.isoformat()` で「2026-06-01」のような文字に。- モデルは `{'id': ..., 'str': ...}` という辞書に（§3.2）。- `changes[field_name] = ...` … 辞書にキー（項目名）と値（変更内容）を登録（§3.2）。- `changes[field_name] = {'old': ..., 'new': ...}` … 更新時は「old（前）」「new（後）」を入れ子の辞書で記録。- `return changes` … 完成した変更辞書を返す。

> **この関数が作るもの（イメージ）**：member の氏名を「admin」→「管理者」に変えると、`get_changes` は次のような辞書を返します。 `python {'name': {'old': 'admin', 'new': '管理者'}}` 「どの項目が、何から何に変わったか」が一目で分かる形です。これが履歴（History）の `changes` 欄に保存されます。

18.4 continue（ループの残りを飛ばす）

§18.3に出てきた `continue` を補足します。

コピー

```
except type(instance).DoesNotExist:
  # この項目は処理せず、次の項目へ
  continue
```

> **用語： `continue`** ループの中で「**今回の繰り返しはここで打ち切り、次の繰り返しへ進む**」命令。`break`（§5.4：ループ自体を抜ける）との違いに注意。`continue` は「この項目だけスキップ、ループは続行」です。

18.5 @receiver で自動登録（111〜142行目）

> **▼ このブロックがやること:** 「member が保存されたら、自動で差分を計算して履歴を残す」関数を登録します。 `@receiver` がその自動化の鍵です（§13.3）。

コピー

```
# 保存前に更新前の値をキャッシュ
# member保存の「直前」に呼ぶ登録（§13.3）
@receiver(pre_save, sender=member)
def cache_old_member_instance(sender, instance, **kwargs):
  # 更新前の値を保管（§18.2）
  set_instance_cache(instance)


# 履歴を記録　新規作成、更新（member）
# member保存の「直後」に呼ぶ登録
@receiver(post_save, sender=member)
def log_create_update_member_history(sender, instance, created, **kwargs):
  # 現在のリクエストを取得
  request = get_current_request()
  # 操作者の番号（三項 §8.1）
  session_data = request.session.get('login_No') if request else None

  # 差分計算
  # §18.3で変更点を計算
  changes = get_changes(instance, created)

  # 操作内容判定
  # 新規ならCREATE、更新ならUPDATE（三項 §8.1）
  operation = 'CREATE' if created else 'UPDATE'

  # 履歴記録
  # 履歴レコードを作成
  History.objects.create(
    # 操作種別
    operation=operation,
    # テーブル名
    table_name='member',
    # 対象レコードのid
    record_id=instance.id,
    # 操作した人の番号
    login_No=session_data,
    # 変更内容（§18.3の辞書）
    changes=changes,
  )
```

流れ： 1. `@receiver(pre_save, sender=member)` … member が保存される **直前**、`cache_old_member_instance` が自動で呼ばれ、更新前の値を保管（§18.2）。2. `@receiver(post_save, sender=member)` … member が保存された **直後**、`log_create_update_member_history` が自動で呼ばれる。3. その中で `get_changes` を呼んで差分を計算（§18.3）。4. `operation = 'CREATE' if created else 'UPDATE'` … 三項演算子（§8.1）で操作種別を決定。5. `History.objects.create(...)` … 履歴を1件作成（誰が・どのテーブルの・どのレコードを・どう変えたか）。

この仕組みの威力: アプリ中のどこで `member` を保存しても、**書き手が意識しなくても** 履歴が自動で残ります。`@receiver` で「保存の合図」に関数を紐づけているからです。signals.py には、member だけでなく Business_Time_graph・team_member・kosu_division など **全モデル分** の同じパターンが並んでいます（だから signals.py は長い）。1つ読めれば全部読めます。

用語： `**kwargs` （クワーグス） 「キーワード引数を、いくつでもまとめて受け取る」書き方。`@receiver` が呼ぶ関数には色々な情報が渡されますが、使わないものは `**kwargs` でまとめて受け流します。「余った引数の受け皿」です。

18.6 削除時の履歴（147〜159行目）

保存（post_save）だけでなく、削除（post_delete）にも `@receiver` が付いています。

コピー

```
# 履歴を記録　削除（member）
# member削除の「直後」に呼ぶ登録
@receiver(post_delete, sender=member)
def log_delete_member_history(sender, instance, **kwargs):
  request = get_current_request()
  session_data = request.session.get('login_No') if request else None

  # 履歴記録
  History.objects.create(
    # 操作は削除
    operation='DELETE',
    table_name='member',
    record_id=instance.id,
    login_No=session_data,
    # 削除なので差分は無し（None）
    changes=None,
  )
```

- `@receiver(post_delete, sender=member)` … member が削除された直後に呼ばれる。
- `operation='DELETE'` … 削除という操作種別。
- `changes=None` … 削除には「前後の差分」がないので None。

保存と削除のペア: 各モデルについて「保存時（CREATE/UPDATE）」と「削除時（DELETE）」の2つの `@receiver` が用意されています。これで作成・更新・削除のすべてが履歴に残り、「いつ・誰が・何をしたか」を後から完全に追跡できます。本アプリの **監査ログ（操作の記録）** の中核です。

19. トラブルシューティング

Pythonの初心者がつまずきやすいエラーと対処をまとめます。エラーメッセージ（英語）の最後の行に「エラーの種類」が出るので、まずそこを見ます。

エラー（種類）	日本語の意味	よくある原因	対処
`IndentationError`	インデントエラー	字下げがズレている／スペースとタブ混在	字下げを揃える（§1）。エディタで「Tabをスペースに」設定
`SyntaxError`	文法エラー	`:` の付け忘れ、カッコの閉じ忘れ	if/for/def/class の行末の `:` を確認（§1）
`NameError`	名前エラー	未定義の変数・スペルミス	変数名のスペル、import 忘れを確認（§12）
`TypeError`	型エラー	`"5" + 5` のように型が合わない演算	`int()` / `str()` で型を揃える（§2.3）
`ValueError`	値エラー	`int("abc")` のような変換失敗	try-except で受ける（§10）。本アプリは対処済み
`KeyError`	キーエラー	辞書に無いキーを `[ ]` で取った	`.get()` を使う（§3.2）
`IndexError`	添字エラー	リストの範囲外を取った（`[10]` が無い）	リストの長さを確認。0始まりに注意（§3.1）
`AttributeError`	属性エラー	無いメソッド・属性を呼んだ	スペルミス、対象の型を確認
`member.DoesNotExist`	データなし	`.get()` で該当データが無い	try-except で受ける（§10.4）。本アプリは対処済み
`ModuleNotFoundError`	モジュールなし	import 先が無い／未インストール	相対importの `.` / `..` を確認（§12.3）、`pip install`

> **エラーの読み方:** Pythonのエラーは長い「トレースバック（traceback）」が出ますが、**一番下の行** が「種類: 説明」です。たとえば `ValueError: invalid literal for int()` なら「数値に変換できない値だった」。種類が分かればこの表で対処できます。その上の行は「どのファイルの何行目で起きたか」なので、そこを開いて直します。

---

20. 演習問題

手を動かすと定着します。**ローカル環境**（壊しても安全）で試してください。Pythonの対話シェル（コマンドで `python` と打つと起動）でも試せます。

1. **変数と型**：`x = "5"` と `y = 5` を作り、`x + x` と `y + y` の結果を予想してから `print()` で確認せよ。なぜ違うか §2.3 を読んで説明できるようにする。
2. **リストと内包表記**：リスト `[1, 2, 3, 4, 5, 6]` から「偶数だけ」を集める内包表記を書け（ヒント：`if n % 2 == 0`。`%` は割った余り）。`main_views.py` の `valid_member_numbers`（§6.2）を参考に。
3. **辞書**：`person = {"name": "田中", "no": 12345}` を作り、`person["name"]` と `person.get("age")` の結果の違いを確認せよ。なぜ `.get()` が安全か §3.2 で確認。
4. **三項演算子**：「点数が60以上なら'合格'、未満なら'不合格'」を、まず if-else で書き、次に三項演算子（§8.1）で1行に書き換えよ。
5. **f文字列**：`name = "admin"`、`count = 3` のとき、`f"..."` を使って「adminさんは3件あります」と表示せよ（§8.2）。
6. **関数**：2つの数を受け取り「大きい方」を返す関数 `bigger(a, b)` を `def` で定義せよ（§7）。三項演算子を使うとさらに短くなる。
7. **try-except**：`int(input())` で数値を受け取り、数字以外が入ったら「数値を入れてください」と表示する処理を書け（§10）。`main_utils.py` の §15.3 を参考に。
8. **コード読解**：`main_views.py` の `Login` クラス（§14.6）を見て、「従業員番号が登録されていなかったとき、どのメッセージが返るか」を答えよ。答え合わせは §14.6 の最後の `else:` で。
9. **クラス**：§9.1 の `Person` クラスを実際に書いて、`tanaka = Person("田中", 30)` を作り `tanaka.greet()` を呼べ。`self`・`__init__` の役割を §9 で再確認。
10. **signals 読解**：`signals.py`（§18.5）で、member の氏名を変えたとき History に残る `changes` が `{'name': {'old': ..., 'new': ...}}` の形になる理由を、§18.3 の `get_changes` の流れで説明せよ。

---

21. この章のまとめ

- Python はバックエンド（サーバー側＝厨房）の言語。本アプリでは `kosu/` フォルダの `.py` ファイル群がそれ。
- **インデント（字下げ）** が処理のまとまりを表す（波カッコの代わり）。`:` で始め、次行から字下げ——これが if/for/def/class/try/with 共通の基本形。本アプリはスペース2個。
- **変数**は `名前 = 値`。**型**は int（整数）・float（小数）・str（文字）・bool（True/False）・None（値なし）。
- **入れ物**は3種：リスト `[ ]`（順番）・辞書 `{ }`（名前:値）・タプル `( )`（変更不可）。
- **if** で条件分岐（`==` 比較・`and/or/not`・truthy/falsy・`is None`）、**for** で繰り返し（`range`・`break`・`continue`・入れ子）。
- **リスト内包表記** `[式 for x in リスト if 条件]` でリストを1行生成（実コードの班員抽出・日付生成）。

- **関数** `def 名前(引数):` … 初期値・複数戻り値（タプル）。**三項演算子** `A if 条件 else B`、**f文字列** `f"{値}"`。
- **クラス**は設計図、**インスタンス**は実体。`self`（自分自身）・`__init__`（初期化）・`__str__`（表示名）・**継承** `(親)`・`super()`・オーバーライド。本アプリのモデルは全て `models.Model` を継承。
- **try-except** でエラーに耐える、**with** でファイルを安全に開閉、**import**（相対 `.` / `..`）で他ファイルの機能を借りる、**デコレータ** `@`（`@api_view`・`@receiver`）で機能を追加・自動化。
- 実コード `main_views.py`（Login/PrintToLogger/get_logs/TeamMenu）・`main_utils.py`（**検証・モデル名収集**）・`asynchronous_views.py`（validate_dates）・`models.py`（member）・`signals.py`（**差分計算と@receiver**） を1行ずつ読み切った。

次は、このPythonで書かれたバックエンドが「Webサーバー」としてどう動くか——URLの受付・データベース操作・APIの仕組み——を学ぶ「07_Django_バックエンド.md」へ進みます。